

# Frontend Backend und Plattformmodule

Komponenteninventar, Design Tokens, Accessibility und Modul-READMEs

Die Implementierungsebene: Frontend-Architektur mit Komponenteninventar, Design Tokens, Performance-Baseline und Accessibility-Audit, dazu die READMEs und Changelogs aller Plattformmodule unter `src/platform`. Hier steht, woraus die Anwendung konkret gebaut ist.

Repository: [SvenKulesa/Finance](#)

Stand: `claude/pdf-dokumente-notebook-llm-l6kcxq @ 4eadf9d`

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 40

Umfang: ca. 12.362 Wörter

---

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

# Inhalt dieses Quellpakets

---

|                                                 |    |
|-------------------------------------------------|----|
| 1. Backend Architecture & Service Guidelines    | 5  |
| docs/backend/BACKEND_ARCH.md                    |    |
| 2. CAPITAL-AI Frontend – Component Inventory    | 7  |
| docs/frontend/COMPONENT_INVENTORY.md            |    |
| 3. Frontend Architecture & Interface Guidelines | 12 |
| docs/frontend/FRONTEND_ARCH.md                  |    |
| 4. CAPITAL-AI Frontend Roadmap                  | 14 |
| docs/frontend/FRONTEND_ROADMAP.md               |    |
| 5. Phase 0 — Accessibility Audit Checklist      | 18 |
| docs/frontend/PHASE0_ACCESSIBILITY_AUDIT.md     |    |
| 6. Phase 0 — Design Tokens                      | 21 |
| docs/frontend/PHASE0_DESIGN_TOKENS.md           |    |
| 7. Phase 0 — Loading & Error State Patterns     | 23 |
| docs/frontend/PHASE0_LOADING_ERROR_STATES.md    |    |
| 8. Phase 0 — Performance Baseline               | 26 |
| docs/frontend/PHASE0_PERFORMANCE_BASELINE.md    |    |
| 9. Phase 0 — StatusBadge Primitive              | 28 |
| docs/frontend/PHASE0_STATUS_BADGE.md            |    |
| 10. Phase 1 — Quick Wins                        | 30 |
| docs/frontend/PHASE1_QUICK_WINS.md              |    |
| 11. Architecture                                | 32 |
| src/platform/Architecture/README.md             |    |
| 12. Compliance                                  | 34 |
| src/platform/Compliance/README.md               |    |
| 13. Contracts                                   | 36 |
| src/platform/Contracts/README.md                |    |
| 14. basisschicht                                | 38 |
| src/platform/Core/basisschicht.md               |    |
| 15. Core                                        | 39 |
| src/platform/Core/README.md                     |    |
| 16. Discovery                                   | 41 |
| src/platform/Discovery/README.md                |    |
| 17. Documentation Governance Validator          | 43 |
| src/platform/Documentary/Governance/README.md   |    |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>18. Documentary</b>                               | <b>47</b> |
| <code>src/platform/Documentary/README.md</code>      |           |
| <b>19. Changelog — EventMesh</b>                     | <b>50</b> |
| <code>src/platform/EventMesh/CHANGELOG.md</code>     |           |
| <b>20. EventMesh</b>                                 | <b>53</b> |
| <code>src/platform/EventMesh/README.md</code>        |           |
| <b>21. Events</b>                                    | <b>57</b> |
| <code>src/platform/Events/README.md</code>           |           |
| <b>22. Generators</b>                                | <b>59</b> |
| <code>src/platform/Generators/README.md</code>       |           |
| <b>23. Interfaces</b>                                | <b>61</b> |
| <code>src/platform/Interfaces/README.md</code>       |           |
| <b>24. Knowledge</b>                                 | <b>63</b> |
| <code>src/platform/Knowledge/README.md</code>        |           |
| <b>25. Models</b>                                    | <b>65</b> |
| <code>src/platform/Models/README.md</code>           |           |
| <b>26. PlatformDirector</b>                          | <b>67</b> |
| <code>src/platform/PlatformDirector/README.md</code> |           |
| <b>27. Plugins</b>                                   | <b>69</b> |
| <code>src/platform/Plugins/README.md</code>          |           |
| <b>28. Quality</b>                                   | <b>71</b> |
| <code>src/platform/Quality/README.md</code>          |           |
| <b>29. Registry</b>                                  | <b>73</b> |
| <code>src/platform/Registry/README.md</code>         |           |
| <b>30. Release</b>                                   | <b>75</b> |
| <code>src/platform/Release/README.md</code>          |           |
| <b>31. Security</b>                                  | <b>79</b> |
| <code>src/platform/Security/README.md</code>         |           |
| <b>32. Shared</b>                                    | <b>81</b> |
| <code>src/platform/Shared/README.md</code>           |           |
| <b>33. Supervisor</b>                                | <b>83</b> |
| <code>src/platform/Supervisor/README.md</code>       |           |
| <b>34. Telemetry</b>                                 | <b>84</b> |
| <code>src/platform/Telemetry/README.md</code>        |           |
| <b>35. Changelog — Traceability</b>                  | <b>87</b> |
| <code>src/platform/Traceability/CHANGELOG.md</code>  |           |
| <b>36. Traceability</b>                              | <b>90</b> |

[src/platform/Traceability/README.md](#)

---

|                       |           |
|-----------------------|-----------|
| <b>37. Validators</b> | <b>95</b> |
|-----------------------|-----------|

[src/platform/Validators/README.md](#)

---

|                           |           |
|---------------------------|-----------|
| <b>38. VersionManager</b> | <b>97</b> |
|---------------------------|-----------|

[src/platform/VersionManager/README.md](#)

---

|                      |           |
|----------------------|-----------|
| <b>39. Changelog</b> | <b>99</b> |
|----------------------|-----------|

[src/platform/Vocabulary/CHANGELOG.md](#)

---

|                                                     |            |
|-----------------------------------------------------|------------|
| <b>40. CAPITAL-AI Canonical Vocabulary Registry</b> | <b>100</b> |
|-----------------------------------------------------|------------|

[src/platform/Vocabulary/README.md](#)

---

# Backend Architecture & Service Guidelines

Quelle: docs/backend/BACKEND\_ARCH.md

## Backend Architecture & Service Guidelines

**Project:** CAPITAL-AI

**Stack:** Node.js / Express / TypeScript / Supabase / PostgreSQL

### Architectural Concept: High-Performance API Gateway

The backend serves as a secure, high-performance API Gateway designed to proxy client requests to external providers (CoinGecko, Stooq, Stripe), enforce row-level security on PostgreSQL tables, verify subscription tiers, and serve static assets.

### Core Operational Directives

#### 1. Robust Server-Side Caching (API Guarding)

To neutralize client-induced rate limiting (such as `429 Too Many Requests` on public APIs like CoinGecko), all external market feeds must use server-side cache layers. \* **Cache TTL:** Market prices cached for **60 seconds** (`MARKET_DATA_CACHE_TTL = 60000`). \* **Request Coalescing:** Combine identical incoming concurrent requests using a shared, active Promise (`activeMarketDataPromise`) to avoid multi-fetching. \* **Dynamic Fallbacks:** In case of complete remote API failure, serve expired cache contents, or fall back to high-fidelity, simulated price fluctuations.

#### 2. Lazy Initialization for Client SDKs

To avoid crashes on startup when keys are missing or invalid, all SDK wrappers (like Stripe and Supabase clients) must be loaded lazily on-demand.

```
let serverSupabaseClient: any = null;

function getServerSupabase() {
  if (!serverSupabaseClient) {
    const url = process.env.SUPABASE_URL || process.env.VITE_SUPABASE_URL;
    const key = process.env.SUPABASE_SERVICE_ROLE_KEY || process.env.SUPABASE_ANON_KEY;
    if (!url || !key) {
      throw new Error("Supabase credentials missing.");
    }
    serverSupabaseClient = createClient(url, key);
  }
  return serverSupabaseClient;
}
```

### 3. Raw Body Webhook Ingestion

The Stripe signature check ( `stripe.webhooks.constructEvent` ) requires the exact, unaltered raw buffer body. Consequently, webhook routing must be defined **before** applying global body-parser middlewares like `express.json()` .

### Stripe Production Integration

The Stripe billing engine supports unbegrenzte, secure payment checkout and subscription cycle management.

#### Key Implementation Guidelines:

- **Keys Protection:** Enforce that the `STRIPE_SECRET_KEY` and the `STRIPE_WEBHOOK_SECRET` are stored purely in server-side process environments.
- **Checkout Sessions:** Dynamic pricing tied directly to environment variables ( `STRIPE_PRICE_ID_STARTER` , `STRIPE_PRICE_ID_PRO` , etc.). Success and cancel URLs must carry the session identifier ( `?session_id={CHECKOUT_SESSION_ID}` ) to verify the payment on return.
- **Idempotent Webhooks:** Secure webhook routing to verify event signatures, map customer metadata to Supabase user IDs, and perform idempotent database updates to prevent multiple subscription allocations.
- **Self-Service Customer Portal:** Expose endpoints where users can upgrade, downgrade, or cancel subscriptions through Stripe's native Billing Portal safely.

### API Routes Matrix

| Endpoint                                   | HTTP | Description                                       | Security              |
|--------------------------------------------|------|---------------------------------------------------|-----------------------|
| <code>/api/market-data</code>              | GET  | Real-time aggregate pricing, volumes, and scores. | Rate Limited / Cached |
| <code>/api/backtest-history</code>         | GET  | Retrieves daily historical closing price tables.  | Parameter Validation  |
| <code>/api/stripe/user-subscription</code> | GET  | Syncs subscription tiers from local tables.       | JWT Token Authorized  |
| <code>/api/stripe/checkout</code>          | POST | Spawns checkout sessions on Stripe Gateway.       | User Authenticated    |
| <code>/billing/webhook</code>              | POST | Direct ingest for Stripe webhook events.          | Signature Verified    |

# CAPITAL-AI Frontend – Component Inventory

Quelle: docs/frontend/COMPONENT\_INVENTORY.md

## CAPITAL-AI Frontend – Component Inventory

**Stand:** 16. August 2026  
**Quelle:** src/components/ (Finance-Repo)  
**Namenskonvention:** PascalCase, Dateiname = Komponententname

Dieses Inventory listet die aktuellen UI-/Feature-Komponenten und dient als Grundlage für Phase 0 der Frontend-Roadmap.

### Design-Primitives (Phase 0+)

| Komponente  | Datei           | Kurzbeschreibung                                                                                           |
|-------------|-----------------|------------------------------------------------------------------------------------------------------------|
| StatusBadge | StatusBadge.tsx | Kanonischer Status-Badge (READY / REJECT / DATA_UNAVAILABLE / LOADING / ...) mit Icon + Text, WCAG-konform |

### Kern-Dashboard & Cockpit

| Komponente                | Datei                         | Kurzbeschreibung                                         |
|---------------------------|-------------------------------|----------------------------------------------------------|
| Dashboard                 | Dashboard.tsx                 | Haupt-Dashboard (Multi-Asset, Scores, Live-Intelligence) |
| AssetUniverseDashboard    | AssetUniverseDashboard.tsx    | Enterprise-Universum-Übersicht                           |
| UniverseBestWorst         | UniverseBestWorst.tsx         | Best/Worst-Ranking im Universum                          |
| Screener                  | Screener.tsx                  | Quantitativer Multi-Asset-Screener                       |
| MarketScreener            | MarketScreener.tsx            | Markt-Screener-Variante                                  |
| Watchlist                 | Watchlist.tsx                 | Persönliche Watchlist                                    |
| FavoriteAssetPatternSlots | FavoriteAssetPatternSlots.tsx | Favoriten-/Pattern-Slots                                 |

### Scoring & Analyse

| Komponente              | Datei                       | Kurzbeschreibung                              |
|-------------------------|-----------------------------|-----------------------------------------------|
| CryptoScoringEnterprise | CryptoScoringEnterprise.tsx | Enterprise-Crypto-Scoring (nutzt StatusBadge) |
| BuffetValueCheck        | BuffetValueCheck.tsx        | Graham/Buffett Value-Check                    |

| Komponente                     | Datei                              | Kurzbeschreibung                |
|--------------------------------|------------------------------------|---------------------------------|
| BacktestEngine                 | BacktestEngine.tsx                 | Backtesting-Engine              |
| PortfolioBacktester            | PortfolioBacktester.tsx            | Portfolio-Backtester            |
| PortfolioPerformance           | PortfolioPerformance.tsx           | Portfolio-Performance           |
| MonteCarloDetailed             | MonteCarloDetailed.tsx             | Monte-Carlo-Detailansicht       |
| RealTimeRiskAssessment         | RealTimeRiskAssessment.tsx         | Echtzeit-Risiko                 |
| EnterpriseAnalysisPanels       | EnterpriseAnalysisPanels.tsx       | Enterprise-Analyse-Panels       |
| EnterpriseBinanceQuickAnalysis | EnterpriseBinanceQuickAnalysis.tsx | Schnellanalyse Binance          |
| LandingBinanceQuickAnalysis    | LandingBinanceQuickAnalysis.tsx    | Landing-Schnellanalyse          |
| HeatmapCreator                 | HeatmapCreator.tsx                 | Heatmap-Erstellung              |
| QuantumGraph                   | QuantumGraph.tsx                   | Graph-Visualisierung            |
| Charts                         | Charts.tsx                         | Chart-Komponenten (Recharts/D3) |
| PerformanceDashboard           | PerformanceDashboard.tsx           | Performance-Übersicht           |
| RawMaterialsDashboard          | RawMaterialsDashboard.tsx          | Rohstoff-Dashboard              |
| DeFiOrchestration              | DeFiOrchestration.tsx              | DeFi-Orchestrierung             |

## Sentiment, News & AI

| Komponente           | Datei                    | Kurzbeschreibung                        |
|----------------------|--------------------------|-----------------------------------------|
| SentimentDashboard   | SentimentDashboard.tsx   | Sentiment-Dashboard                     |
| MarketSentiment      | MarketSentiment.tsx      | Markt-Sentiment                         |
| RealtimeAiNewsfeed   | RealtimeAiNewsfeed.tsx   | AI-Newsfeed                             |
| Newsticker           | Newsticker.tsx           | Newsticker                              |
| MarkdownOrchestrator | MarkdownOrchestrator.tsx | Markdown-/Dokumentations-Orchestrierung |
| OrchestratorPanel    | OrchestratorPanel.tsx    | Orchestrator-Panel                      |
| InteractModule       | InteractModule.tsx       | Interaktionsmodul                       |
| ImageAnalyzer        | ImageAnalyzer.tsx        | Bildanalyse                             |

## Auth, Profile, Billing

| Komponente      | Datei               | Kurzbeschreibung   |
|-----------------|---------------------|--------------------|
| LoginStepUpGate | LoginStepUpGate.tsx | Step-up Login-Gate |

| Komponente                 | Datei                          | Kurzbeschreibung         |
|----------------------------|--------------------------------|--------------------------|
| StepUpModal                | StepUpModal.tsx                | Step-up Modal            |
| RegistrationCompletionGate | RegistrationCompletionGate.tsx | Registrierungs-Abschluss |
| ProfilePage                | ProfilePage.tsx                | Profilseite              |
| PasskeySettings            | PasskeySettings.tsx            | Passkey-Einstellungen    |
| TotpSettings               | TotpSettings.tsx               | TOTP-Einstellungen       |
| Abonnements                | Abonnements.tsx                | Abonnement-Übersicht     |
| SubscriptionModal          | SubscriptionModal.tsx          | Abo-Modal                |
| Checkout                   | Checkout.tsx                   | Checkout                 |
| GuestCliffhangerModal      | GuestCliffhangerModal.tsx      | Guest-Cliffhanger        |

## Compliance, Admin, Governance

| Komponente                | Datei                         | Kurzbeschreibung              |
|---------------------------|-------------------------------|-------------------------------|
| ComplianceBadge           | ComplianceBadge.tsx           | Compliance-Badge              |
| ComplianceConsentModal    | ComplianceConsentModal.tsx    | Consent-Modal                 |
| ComplianceExporter        | ComplianceExporter.tsx        | Compliance-Export             |
| ComplianceNotifications   | ComplianceNotifications.tsx   | Compliance-Benachrichtigungen |
| SecurityComplianceAuditor | SecurityComplianceAuditor.tsx | Security/Compliance-Auditor   |
| SecurityRadarBadge        | SecurityRadarBadge.tsx        | Security-Radar-Badge          |
| AuditLog                  | AuditLog.tsx                  | Audit-Log                     |
| AuditLogs                 | AuditLogs.tsx                 | Audit-Logs-Übersicht          |
| AuditLogManager           | AuditLogManager.tsx           | Audit-Log-Manager             |
| AdminPanel                | AdminPanel.tsx                | Admin-Panel                   |
| AdminPortal               | AdminPortal.tsx               | Admin-Portal                  |
| SupervisorDashboard       | SupervisorDashboard.tsx       | Supervisor-Dashboard          |
| DocumentHygienePanel      | DocumentHygienePanel.tsx      | Dokumenten-Hygiene            |
| VersionManagerPanel       | VersionManagerPanel.tsx       | Version-Manager               |
| AdrForm                   | AdrForm.tsx                   | ADR-Formular                  |
| AuthStateDebugger         | AuthStateDebugger.tsx         | Auth-State-Debugger           |
| SystemLatencyMonitor      | SystemLatencyMonitor.tsx      | Latenz-Monitor                |
| SeoDashboard              | SeoDashboard.tsx              | SEO-Dashboard                 |

## Landing, Legal, Branding, Utils

| Komponente                 | Datei                          | Kurzbeschreibung          |
|----------------------------|--------------------------------|---------------------------|
| LandingPage                | LandingPage.tsx                | Landing Page              |
| CapitalAiLogo              | CapitalAiLogo.tsx              | Logo                      |
| AssetLogo                  | AssetLogo.tsx                  | Asset-Logos               |
| Datenschutz                | Datenschutz.tsx                | Datenschutz               |
| ImpressumAgb               | ImpressumAgb.tsx               | Impressum / AGB           |
| PdfExportModal             | PdfExportModal.tsx             | PDF-Export-Modal          |
| PriceAlert                 | PriceAlert.tsx                 | Preis-Alerts              |
| SocialAccountManager       | SocialAccountManager.tsx       | Social-Account-Verwaltung |
| SocialDirectPublisherModal | SocialDirectPublisherModal.tsx | Social-Publisher-Modal    |
| ErrorBoundary              | ErrorBoundary.tsx              | Error Boundary            |

## Design-Tokens / Patterns (aus `src/index.css` / Architektur)

| Token / Pattern                                  | Quelle                              | Bemerkung                                                       |
|--------------------------------------------------|-------------------------------------|-----------------------------------------------------------------|
| <code>--color-background / #18181b</code>        | <code>index.css</code>              | Canvas                                                          |
| <code>--color-aif-gold-*</code>                  | <code>index.css</code>              | Premium-Highlight                                               |
| <code>--color-aif-neon-cyan / neon-purple</code> | <code>index.css</code>              | Neural / Accent                                                 |
| Glassmorphism Card                               | <code>FRONTEND_ARCH.md</code>       | <code>bg-neutral-950/40 border-white/10 backdrop-blur-md</code> |
| Focus Outline                                    | <code>index.css</code>              | <code>*:focus-visible gold</code>                               |
| Fonts                                            | Poppins, Montserrat, JetBrains Mono | <code>--font-sans / display / mono</code>                       |
| StatusBadge tones                                | <code>StatusBadge.tsx</code>        | emerald / rose / amber / gold / muted                           |

## Hinweise zur Weiterentwicklung

1. Neue UI-Komponenten in `src/components/` anlegen und hier dokumentieren.
2. Wiederverwendbare Primitives (Button, Badge, Card, Gauge, Chip) schrittweise extrahieren (Phase 1).
3. StatusBadge ist der erste formalisierte Primitive — weitere Call-Sites in Phase 1.
4. Dieses Inventory ist die Basis für Storybook und die Frontend-Roadmap Phase 0.

Erstellt am 16.08.2026 im Rahmen der Frontend-Roadmap. StatusBadge ergänzt  
16.08.2026.

# Frontend Architecture & Interface Guidelines

Quelle: docs/frontend/Frontend\_ARCH.md

## Frontend Architecture & Interface Guidelines

**Project:** CAPITAL-AI  
**Framework:** React 19 / Vite 6 / Tailwind CSS 4 / Motion

### Architectural Concept: Event-Driven SPA

The frontend is constructed as an **Event-Driven Single Page Application (SPA)** that runs within isolated, secure browser contexts. We combine deep cyber-slate aesthetics with strict performance, responsiveness, and accessibility guidelines.

### Visual Identity & Glassmorphism Design System

#### 1. Cyber Slate Aesthetic & Palette

- **Main Canvas Background:** Deep, rich absolute black ( `bg-black` ) to ensure maximum visual contrast and legibility under varying lighting conditions.
- **Glassmorphism Panels:** Interactive modules, cards, and drawers are rendered as semi-transparent dark slate backings styled with fine border borders: `tailwind bg-neutral-950/40 border border-white/10 backdrop-blur-md rounded-xl p-6`
- **Branding Highlights:**
  - **Cyber Green (Success):** Reflects validated parameters, stable setups, or favorable scoring states ( `text-green-400` ).
  - **AIF Gold (Premium Highlight):** Used strictly for premium features, high-priority scoring tiers, and focal calls-to-action ( `text-aif-gold-DEFAULT` ).
  - **Neon Purple / Cyan:** Abstract background neural vector paths to reinforce the high-performance AI engine concept.

#### 2. Kinetic Interaction Design (Motion Guidelines)

Every transition, page load, or button interaction must feel responsive, organic, and fluid. \* Use `motion` for staggered listings, fade-ins, and drawer slide-overs. \* Keep exit animation times tight ( `duration: 0.2` ) to maintain a snappy, high-speed UX feel.

### Directory Structure Standards

The codebase enforces a highly modular, decoupled structure:

```
/src
├── main.tsx           # Clean React 19 mounting and routing entry
├── App.tsx            # Main layout controller and session context synchronizer
├── index.css          # Tailwind CSS 4 directives, custom system-font rules
├── types.ts           # Unified type, interface, and enum declarations
├── hooks/             # Reactive state hooks (e.g. useSubscription)
├── components/        # Independent, single-purpose, isolated UI nodes
│   ├── Screener.tsx   # Real-time quantitative stock screener
│   ├── BacktestEngine.tsx # Interactive portfolio backtesting mask
│   ├── BuffetValueCheck.tsx # Benjamin Graham DCF calculator
│   └── MarkdownOrchestrator.tsx # Interactive documentation & compliance hub
```

## Mobile-First Desktop Precision (BFSG Accessibility)

- **Hit Target Sizing:** In accordance with German **BFSG** and European **EN 301 549** standards, all touch-sensitive navigation items, action chips, and buttons must have a minimum tap area of **44x44px**.
- **Responsive Reflows:** Enforce mobile-first responsive grid layouts (e.g., `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3`) to guarantee perfect responsiveness across mobile smartphones, tablets, and 4K desktop screens.
- **Reactive Size Handling:** Never hardcode dimensions for charts, tables, or canvases. Always bind them to parent nodes using `ResizeObservers` or responsive layouts to adapt instantly to viewport changes.

# CAPITAL-AI Frontend Roadmap

---

Quelle: docs/frontend/Frontend\_ROADMAP.md

## CAPITAL-AI Frontend Roadmap

**Projekt:** capital-ai.online

**Repository:** SvenKulesa/Finance

**Version:** 1.3

**Stand:** 16. August 2026

**Owner:** Sven Kulesa / Capital-AI

**Bezug:** docs/frontend/Frontend\_ARCH.md , src/index.css (@theme), public mirror: SvenKulesa/capital-ai-frontend

Dieses Dokument ist die **kanonische Frontend-Roadmap** für die Weiterentwicklung des React-19-Clients hin zu einem State-of-the-Art Design- und UX-Niveau (2026).

---

### 1. Vision

Das Frontend soll sich von einem funktionalen, dichten Dashboard zu einem **ruhigen, präzisen und institutionell wirkenden Cockpit** entwickeln:

- Klare visuelle Hierarchie
- Reduzierte kognitive Last
- Exzellente Data-Visualisierung (Recharts / D3)
- AI-native Interaktionen (Explainability, Conversational Layer)
- Hohe Accessibility (WCAG 2.2 AA / BFG / EN 301 549) & Performance
- Konsistentes Design-System auf Basis der bestehenden Cyber-Slate / Glassmorphism-Identität

Zielbild: Vergleichbar mit modernen FinTech-/Quant-Interfaces (Linear-ähnliche Präzision + TradingView-Datenqualität + AI-Conversational Layer).

---

### 2. Aktueller Stand (Kurzbewertung)

**Stärken** - Starke thematische Dark-Theme-Identität ( #18181b , AIF-Gold, Neon-Cyan/Purple) - Klare Status-Kommunikation (READY / REJECT / DATA\_UNAVAILABLE) — via `StatusBadge` - Gute Grundstruktur für Multi-Asset-Scoring ( `Dashboard` , `AssetUniverseDashboard` , `Screeener` , `CryptoScoringEnterprise` ) - Responsive Basis und Motion-Integration vorhanden - Fokus-Outline, Reduced-Motion und Glassmorphism-Patterns in `index.css` / `Frontend_ARCH.md`

**Schwächen** - Hohe Informationsdichte in großen Komponenten (z. B. `Dashboard.tsx` ) - Card-Hierarchie und Whitespace noch nicht überall konsistent - Score-Visualisierungen und

Multi-Faktor-Matrix können noch prominenter werden - Fehlendes starkes Onboarding & progressive Disclosure - Kein Storybook / Component-Library-Dokumentation

---

### 3. Roadmap-Phasen

#### Phase 0 - Fundament (Woche 1-2) — DONE (Live-Baseline optional pending)

**Ziel:** Stabile Basis schaffen

- [x] Design-Tokens formalisieren — `docs/frontend/design-tokens.json` + `PHASE0_DESIGN_TOKENS.md`
- [x] Accessibility-Audit-Checkliste — `PHASE0_ACCESSIBILITY_AUDIT.md` (Live-Messung pending)
- [x] Performance-Baseline-Protokoll — `PHASE0_PERFORMANCE_BASELINE.md` (Messung pending)
- [x] Einheitliche Error- und Loading-States spezifiziert — `PHASE0_LOADING_ERROR_STATES.md`
- [x] Component Inventory finalisieren — `docs/frontend/COMPONENT_INVENTORY.md`
- [x] StatusBadge-Primitive + Call-Sites — `src/components/StatusBadge.tsx`
- [ ] Storybook-Grundlage vorbereiten (optional, parallel)
- [ ] Live Lighthouse / axe-Messung dokumentieren

**Deliverables:** Tokens, Specs, Inventory, StatusBadge

---

#### Phase 1 - Visual Design System & Look & Feel (Woche 3-6) — IN PROGRESS

**Ziel:** Professionelles, ruhiges Erscheinungsbild

- [ ] Dark-Theme verfeinern (präzisere Semantic Colors auf Basis AIF-Gold / Neon)
- [x] Mehr Whitespace und klarere Card-Hierarchie — Utilities + Enterprise-Scorer (siehe `PHASE1_QUICK_WINS.md`)
- [x] Hintergrund-Partikel / Neural-Animationen abschwächen + `prefers-reduced-motion`
- [x] Einheitliche Badge-Stile (StatusBadge); Card-/Button-Primitive noch offen
- [ ] Score-Gauges und Multi-Faktor-Matrix visuell vervollständigen und prominent platzieren
- [ ] Typografie-Upgrade (bestehende Poppins / Montserrat / JetBrains Mono nutzen und Tracking optimieren)
- [ ] Icon-Library (lucide-react) vereinheitlichen

**Deliverables:**

`PHASE1_QUICK_WINS.md`, CSS-Utilities, Enterprise Spacing/Hit-Targets · weitere Surfaces

---

## Phase 2 – Information Architecture & UX (Woche 5-10)

**Ziel:** Klare Struktur und geführte Nutzung

- [ ] Progressive Disclosure einführen
- [ ] Primäre Hierarchie: Score → Kurzanalyse → Detail-Matrix
- [ ] Verbesserte Asset-Suche & Filter-UX
- [ ] Onboarding / First-Time-User-Flow
- [ ] Empty States, Skeleton Loaders, Success/Error Feedback
- [ ] Secondary Navigation überarbeiten (weniger parallele CTAs)
- [ ] Command-Palette (Power-User) vorbereiten

**Deliverables:** UX-Flows, Wireframes, aktualisierte Navigation

---

## Phase 3 – Interaktionen & Data Visualization (Woche 8-14)

**Ziel:** Polierte Interaktionen und starke Visualisierungen

- [ ] Mikro-Interaktionen und sanfte Transitions (Motion)
- [ ] Interaktive Multi-Faktor-Bewertungsmatrix
- [ ] Verbesserte Score-Visualisierungen (Gauge, Trend, Breakdown) mit Recharts / D3
- [ ] Keyboard-Navigation und Focus-Management
- [ ] Live-Daten-Feedback (subtil)
- [ ] Export- und Share-Funktionen für Scores

**Deliverables:** Interaktive Komponenten, Data-Viz Specs

---

## Phase 4 – Mobile & Accessibility (parallel ab Woche 4)

**Ziel:** Exzellente mobile und barrierefreie Erfahrung

- [x] Touch-optimierte Chips (44×44) — Enterprise-Filter/Timeframe + `.ui-hit`
- [ ] Mobile Informationsarchitektur optimieren
- [ ] Vollständige Screenreader-Unterstützung
- [ ] Kontrast- und Fokus-Optimierung
- [x] Reduced-Motion Support ( `prefers-reduced-motion` in `index.css` )

**Deliverables:** Mobile Specs, Accessibility-Checklist (grün)

---

## Phase 5 – AI-native Features & Skalierung (ab Monat 4+)

**Ziel:** Zukunftssicher und AI-first

- [ ] Conversational Layer (Chat über Scores & Analysen)

- [ ] Personalisierte Dashboards / Saved Views
- [ ] Explainability-UI („Warum dieser Score?“)
- [ ] Vollständiges Design-System + Storybook
- [ ] Komponenten-Bibliothek dokumentieren
- [ ] Theming & mögliche Light-Mode-Option (optional)

**Deliverables:** AI-Chat-Interface, personalisierte Views, Design-System v1.0

---

## 4. Quick Wins

1. ~~Mehr Abstand zwischen den Hauptkarten~~ → gestartet (Enterprise + CSS)
2. Score- und Matrix-Bereiche vollständig sichtbar und prominent machen
3. ~~Einheitliche Status-Badges~~ → StatusBadge live
4. ~~Mobile Chip-Layout und Button-Größen (BFSG)~~ → `.ui-hit` + Enterprise
5. ~~Hintergrund / Neural-Pulse abschwächen~~ → Keyframes + reduced-motion
6. Klare primäre Aktion pro Viewport definieren

Details: `docs/frontend/PHASE1_QUICK_WINS.md`

---

## 5. Erfolgsmetriken

- Lighthouse Performance  $\geq 90$
  - Accessibility Score  $\geq 95$
  - Reduktion der Time-to-First-Score
  - Positive Nutzer-Feedback zu Klarheit und Übersichtlichkeit
  - Komponenten-Wiederverwendbarkeit  $> 80\%$
- 

## 6. Nächste Schritte

1. ~~Phase 0 + StatusBadge~~ → done
  2. Phase 1 Quick Wins ausweiten ( `Dashboard` , `AssetUniverseDashboard` , `MarketScreener` )
  3. Live Lighthouse / axe-Baseline eintragen
  4. Regelmäßige Reviews (alle 2 Wochen)
  5. Öffentlicher Mirror `SvenKulesa/capital-ai-frontend` bei Bedarf synchron halten
- 

Dokument erstellt am 16.08.2026 – Phase 0 done / Phase 1 gestartet 16.08.2026.

# Phase 0 — Accessibility Audit Checklist

Quelle: docs/frontend/PHASE0\_ACCESSIBILITY\_AUDIT.md

## Phase 0 — Accessibility Audit Checklist

**Status:** BASELINE / IN PROGRESS  
**Stand:** 16. August 2026  
**Ziel:** WCAG 2.2 AA · BFSG · EN 301 549  
**Bezug:** FRONTEND\_ARCH.md , src/index.css ( `*:focus-visible` )

### 1. Bereits vorhanden (Code-Evidence)

| Kontrolle              | Evidence                                                                         | Status                        |
|------------------------|----------------------------------------------------------------------------------|-------------------------------|
| Fokus-Outline          | <code>*:focus-visible { outline: 2px solid #F5C453; outline-offset: 4px }</code> | PASS (Basis)                  |
| Min. Hit-Target Policy | FRONTEND_ARCH: 44×44 px dokumentiert                                             | DOCUMENTED — Umsetzung prüfen |
| Responsive Grids       | <code>grid-cols-1 md:... lg:...</code> Pattern in Architektur                    | PARTIAL                       |
| Dark Contrast Base     | <code>#18181b</code> + weißer Text                                               | Basis ok — Komponenten prüfen |

### 2. Audit-Checkliste (manuell / Tools)

#### 2.1 Wahrnehmung (Perceivable)

- [ ] Kontrast Text/Hintergrund  $\geq 4.5:1$  (Normaltext),  $\geq 3:1$  (Large Text / UI)
- [ ] Gold auf Dark ( `#F5C453` auf `#18181b` ) verifizieren
- [ ] Statusfarben (grün/rot/amber) nicht allein farbcodiert — Icon/Text ergänzen
- [ ] Charts: nicht nur Farbe (Pattern/Label)
- [ ] Bilder/Logos: `alt`-Texte ( `AssetLogo` , `CapitalAiLogo` )

#### 2.2 Bedienbarkeit (Operable)

- [ ] Alle interaktiven Elemente per Tastatur erreichbar
- [ ] Sichtbarer Fokus auf allen Controls (Modals, Chips, Tabs)
- [ ] Hit-Targets  $\geq 44 \times 44$  px (Mobile + Desktop Touch)
- [ ] Keine Tastaturfalle in Modals ( `StepUpModal` , `SubscriptionModal` , `ComplianceConsentModal` )
- [ ] Skip-Link oder äquivalente Landmark-Navigation (optional Phase 4)

## 2.3 Verständlichkeit (Understandable)

- ☐ Form-Labels (Auth, Profile, Checkout)
- ☐ Fehlermeldungen programmatisch verknüpft ( `aria-describedby` / `role="alert"` )
- ☐ Sprache der Seite ( `lang="de"` / dynamisch)

## 2.4 Robustheit (Robust)

- ☐ Semantische HTML-Rollen (button vs. div-onClick)
- ☐ ARIA nur wo nötig und korrekt
- ☐ Screenreader-Smoke-Test: Dashboard, Screener, Score-Card, Modal

## 2.5 Motion

- ☐ `prefers-reduced-motion` : Neural-Pulse, Gold-Pulse, Brand-Border dämpfen/abschalten
- ☐ Exit-Animationen  $\leq 0.2s$  beibehalten, aber abschaltbar

## 3. Priorisierte Fundstellen (erste Stichprobe)

| Bereich                                                                                  | Risiko                    | Nächster Schritt                   |
|------------------------------------------------------------------------------------------|---------------------------|------------------------------------|
| Große Composite-Komponenten ( <code>Dashboard.tsx</code> , <code>AdminPanel.tsx</code> ) | Dichte UI, viele Controls | Fokus-Reihenfolge & Hit-Targets    |
| Status-Badges READY/REJECT/DATA_UNAVAILABLE                                              | Nur Farbe                 | Badge-Primitive mit Icon + Text    |
| Charts ( <code>Charts.tsx</code> , Recharts/D3)                                          | Farbabhängigkeit          | Labels / Patterns                  |
| Modals                                                                                   | Fokus-Trap                | Explizite Focus-Management-Prüfung |
| Chips / Filter                                                                           | Mobile Hit-Target         | 44px min height/width              |

## 4. Empfohlene Tools (Baseline messen)

1. Chrome Lighthouse → Accessibility Score (Ziel  $\geq 95$ )
2. axe DevTools / WAVE auf <https://capital-ai.online> (auth + guest)
3. Tastatur-only Walkthrough (Tab / Shift+Tab / Enter / Esc)
4. Optional: VoiceOver / NVDA Smoke-Test

## 5. Abnahme Phase 0 (A11y)

- ☒ Checkliste dokumentiert
- ☐ Lighthouse Accessibility Score gemessen und notiert

- [ ] Kritische Failures (Fokus, Kontrast, Hit-Target) gelistet
- [ ] Keine neuen BLOCKER ohne Ticket/Backlog-Eintrag

**Hinweis:** Vollständige VERIFIED PASS erst nach Live-Messung + Fixes in Phase 1/4.

# Phase 0 — Design Tokens

Quelle: docs/frontend/PHASE0\_DESIGN\_TOKENS.md

## Phase 0 — Design Tokens

**Status:** IN PROGRESS  
**Stand:** 16. August 2026  
**Quelle:** src/index.css ( @theme ), docs/frontend/FRONTEND\_ARCH.md  
**Artefakt:** docs/frontend/design-tokens.json

### Zweck

Formale Extraktion und Erweiterung der Design-Tokens als Single Source of Truth für Phase 0-1. Runtime bleibt Tailwind CSS 4 @theme in src/index.css ; JSON dient Dokumentation, Audits und künftigem Storybook.

### Bereits im Code vorhanden ( src/index.css )

| Token                   | Wert                           | Verwendung          |
|-------------------------|--------------------------------|---------------------|
| --color-background      | #18181b                        | Canvas              |
| --color-foreground      | #FFFFFF                        | Primärtext          |
| --color-border          | rgba(255,255,255,0.08)         | Borders             |
| --color-aif-gold-*      | light / DEFAULT / dark / muted | Premium, Fokus, CTA |
| --color-aif-neon-cyan   | #0DDDDD                        | Neural / Live       |
| --color-aif-neon-purple | #B026FF                        | Accent              |
| --font-sans             | Poppins                        | Body                |
| --font-display          | Montserrat                     | Headings            |
| --font-mono             | JetBrains Mono                 | Scores, Code        |
| Focus ring              | 2px solid #F5C453 , offset 4px | *:focus-visible     |

### Neu formalisiert (Phase 0 — dokumentiert, noch nicht alle in @theme )

#### Semantic Status Colors

| Status | Farbe               | Intent                    |
|--------|---------------------|---------------------------|
| READY  | #4ade80 (green-400) | Verifiziert / freigegeben |
| REJECT | #f87171 (red-400)   | Abgelehnt / Fail          |

| Status           | Farbe                  | Intent                 |
|------------------|------------------------|------------------------|
| OBSERVE          | #fbbf24 (amber-400)    | Beobachten / Grenzfall |
| DATA_UNAVAILABLE | rgba(255,255,255,0.35) | Keine Daten            |

## Spacing & Radius (Empfehlung Phase 1)

- Card padding: 1.5rem ( p-6 )
- Inter-card gap (Whitespace-Quick-Win): 2rem ( gap-8 )
- Primary radius: 0.75rem ( rounded-xl )

## Motion

- Exit / snappy: 0.2s (laut FRONTEND\_ARCH)
- Standard easing: cubic-bezier(0.4, 0, 0.6, 1)
- Reduced-motion: prefers-reduced-motion respektieren (Phase 4 / parallel)

## Glassmorphism Card (kanonisches Pattern)

```
bg-neutral-950/40 border border-white/10 backdrop-blur-md rounded-xl p-6
```

## Nächste Code-Schritte (optional, eigener PR)

1. Semantic status colors in @theme aufnehmen ( --color-status-ready etc.)
2. Spacing-Tokens für section/card in @theme spiegeln
3. prefers-reduced-motion Utility für Neural-/Gold-Animationen

## Abnahme Phase 0 (Tokens)

- [x] Tokens aus index.css inventarisiert
- [x] design-tokens.json angelegt
- [x] Semantic Status + Spacing dokumentiert
- [ ] Optional: @theme -Erweiterung in Runtime-PR

# Phase 0 — Loading & Error State Patterns

Quelle: docs/frontend/PHASE0\_LOADING\_ERROR\_STATES.md

## Phase 0 — Loading & Error State Patterns

**Status:** SPEC READY + REFERENCE IMPLEMENTED  
**Stand:** 16. August 2026  
**Ziel:** Einheitliche UX für Loading, Empty, Error und DATA\_UNAVAILABLE

### 1. State-Modell

| State            | Wann                          | UI-Intent                                           |
|------------------|-------------------------------|-----------------------------------------------------|
| idle             | Noch keine Anfrage            | Placeholder / CTA                                   |
| loading          | Request läuft                 | Skeleton oder subtiler Indicator — kein Layout-Jump |
| ready            | Daten verifiziert             | Score / Content                                     |
| partial          | Teilweise Daten               | Content + Warnhinweis                               |
| data_unavailable | Provider/Daten fehlen         | Neutrales Badge + Erklärung, kein Fake-Score        |
| error            | Technischer Fehler            | Fehlertext + Retry, fail-closed                     |
| reject           | Business-Entscheidung negativ | Status-Badge REJECT, nachvollziehbar                |

**Direktive (AGENTS.md):** Keine Fake-/Mock-Daten an Endnutzer. DATA\_UNAVAILABLE und Errors müssen klar unterscheidbar sein.

### 2. Visuelle Konventionen (an Tokens gebunden)

| State            | Farbe / Pattern                                                       | Badge-Text       |
|------------------|-----------------------------------------------------------------------|------------------|
| READY            | semantic.success #4ade80                                              | READY            |
| REJECT           | semantic.danger #f87171                                               | REJECT           |
| OBSERVE          | semantic.warning #fbbf24                                              | OBSERVE          |
| DATA_UNAVAILABLE | semantic.muted / status.dataUnavailable                               | DATA_UNAVAILABLE |
| ERROR            | danger + role="alert"                                                 | kontextabhängig  |
| LOADING          | Skeleton ( animate-pulse auf Glass-Flächen) oder Spinner in Gold/Cyan | —                |

Badge-Mindestinhalt: **Farbe + Text** (nicht nur Farbe) — A11y.

---

### 3. Skeleton-Pattern (Empfehlung)

Container: gleiche Glass-Card-Struktur wie Ready-State  
Blöcke: `bg-white/5 rounded-md animate-pulse`  
Höhe: typische Score-/Textzeilen nachbilden (CLS vermeiden)  
Dauer: bis Daten oder Error/Unavailable

Reduced-motion: `animate-pulse` durch statischen Placeholder ersetzen.

---

### 4. Error-Pattern

1. Kurze, verständliche Meldung (kein Stack-Trace an Endnutzer)
  2. Optional: Request-/Evidence-ID für Support (ohne PII)
  3. Primäraktion: **Erneut versuchen**
  4. Sekundär: Support / Status-Seite falls vorhanden
  5. `role="alert"` oder live-region für Screenreader
- 

### 5. DATA\_UNAVAILABLE-Pattern

1. Kein numerischer Score vortäuschen
  2. Badge `DATA_UNAVAILABLE`
  3. Ein Satz: warum (z. B. Provider, Asset-Typ, Berechtigung)
  4. Optional: welche Faktoren fehlen (ohne interne Secrets)
- 

### 6. Implementierungs-Reihenfolge (nach Phase 0 Spec)

1. Gemeinsame Badge-Primitive (StatusBadge)
  2. Shared Skeleton-Bausteine für Score-Cards
  3. Vereinheitlichung in `Dashboard`, `AssetUniverseDashboard`, `Screener`, `CryptoScoringEnterprise` (teilweise: `CryptoScoringEnterprise`, `MarketScreener`, `UniverseBestWorst`)
  4. Empty States für Watchlist / Search
- 

### 7. Abnahme Phase 0

- [x] State-Modell dokumentiert
- [x] Visuelle Konventionen an Tokens gekoppelt
- [x] Mindestens eine Referenz-Implementierung (StatusBadge) in Code-PR — `CryptoScoringEnterprise`, `MarketScreener`, `UniverseBestWorst`

- [ ] Inventory der aktuellen ad-hoc Loading/Error-Stellen (Stichprobe)

# Phase 0 — Performance Baseline

Quelle: docs/frontend/PHASE0\_PERFORMANCE\_BASELINE.md

## Phase 0 — Performance Baseline

**Status:** PROCEDURE READY / MEASUREMENT PENDING  
**Stand:** 16. August 2026  
**Zielmetrik (Roadmap):** Lighthouse Performance  $\geq 90$  · Core Web Vitals grün

### 1. Messumgebung

| Parameter | Vorgabe                                                                            |
|-----------|------------------------------------------------------------------------------------|
| URL       | <code>https://capital-ai.online</code> (Landing + authentifzierter Dashboard-Pfad) |
| Tool      | Chrome Lighthouse (Desktop + Mobile), optional PageSpeed Insights                  |
| Netzwerk  | Default Lighthouse throttling                                                      |
| Build     | Produktions-Build ( <code>npm run build</code> / deployed Render)                  |
| Browser   | Aktuelles Chrome, Clean Profile                                                    |

### 2. Core Web Vitals — Zielwerte

| Metrik                 | Gut                  | Ziel Phase 0           |
|------------------------|----------------------|------------------------|
| LCP                    | $\leq 2.5\text{ s}$  | messen & dokumentieren |
| INP                    | $\leq 200\text{ ms}$ | messen & dokumentieren |
| CLS                    | $\leq 0.1$           | messen & dokumentieren |
| TTFB                   | orientierend         | notieren               |
| Lighthouse Performance | $\geq 90$            | Ziel                   |

### 3. Baseline-Protokoll (ausfüllen nach Messung)

#### 3.1 Landing / Public

| Metrik            | Desktop | Mobile | Datum | Notes |
|-------------------|---------|--------|-------|-------|
| Performance Score |         |        |       |       |
| LCP               |         |        |       |       |
| INP               |         |        |       |       |

| Metrik       | Desktop | Mobile | Datum | Notes |
|--------------|---------|--------|-------|-------|
| CLS          |         |        |       |       |
| TBT          |         |        |       |       |
| Bundle hints |         |        |       |       |

### 3.2 Authenticated Dashboard (Hauptcockpit)

| Metrik                   | Desktop | Mobile | Datum | Notes           |
|--------------------------|---------|--------|-------|-----------------|
| Performance Score        |         |        |       |                 |
| LCP                      |         |        |       |                 |
| INP                      |         |        |       |                 |
| CLS                      |         |        |       |                 |
| Time-to-First-Score (UX) |         |        |       | manuell stoppen |

## 4. Bekannte technische Hebel (aus Stack)

| Hebel              | Hinweis                                                                       |
|--------------------|-------------------------------------------------------------------------------|
| React 19 + Vite 6  | Code-Splitting prüfen ( <code>Dashboard</code> / schwere Panels lazy?)        |
| Recharts / D3      | Schwere Viz nicht im initialen Critical Path                                  |
| Motion             | Reduced-motion + weniger permanente Loops (Neural Pulse)                      |
| Fonts              | Google Fonts: <code>display=swap</code> bereits in Import — Latenz beobachten |
| Bilder / AssetLogo | Caching, Größen, lazy                                                         |
| API-Wasserfälle    | Time-to-First-Score = Frontend + Backend-Orchestrierung                       |

## 5. Phase-0-Abnahme

- ☒ Messprotokoll definiert
- ☐ Mindestens eine Desktop- + eine Mobile-Messung dokumentiert
- ☐ Top-3 Performance-Findings priorisiert
- ☐ Keine Produktionsmutation ohne Owner-Freigabe

Messung kann Owner-seitig oder in einem späteren Evidence-PR nachgezogen werden.

# Phase 0 — StatusBadge Primitive

Quelle: docs/frontend/PHASE0\_STATUS\_BADGE.md

## Phase 0 — StatusBadge Primitive

**Status:** IMPLEMENTED (multiple call sites)  
**Stand:** 16. August 2026  
**Code:** src/components/StatusBadge.tsx  
**Spec:** docs/frontend/PHASE0\_LOADING\_ERROR\_STATES.md

### API

```
import { StatusBadge } from './StatusBadge';  
  
<StatusBadge status="READY" />  
<StatusBadge status="DATA_UNAVAILABLE" />  
<StatusBadge status="LOADING" />  
<StatusBadge status={result.status} label="Custom" size="md" />
```

### Supported statuses

| Status               | Tone        | Icon           |
|----------------------|-------------|----------------|
| READY                | emerald     | CheckCircle2   |
| REJECT               | rose        | XCircle        |
| DATA_UNAVAILABLE     | muted white | Database       |
| LOADING              | gold        | Loader2 (spin) |
| SCORE_NOT_COMPUTABLE | amber       | CircleSlash    |
| OBSERVE              | orange      | AlertTriangle  |
| ERROR                | red         | AlertTriangle  |

Unknown strings fall back to amber + normalized label. Soft aliases map NOT\_COMPUTABLE → DATA\_UNAVAILABLE, etc.

### Accessibility

- role="status" + aria-label
- Icon **and** text (not color-only)
- Font-mono uppercase for scanability

## Adoption

| Location                                | Status          |
|-----------------------------------------|-----------------|
| CryptoScoringEnterprise (header status) | wired           |
| MarketScreener (result card status)     | wired           |
| UniverseBestWorst (unavailable rows)    | wired           |
| Dashboard / Screener / other panels     | backlog Phase 1 |

## Next

1. Replace remaining ad-hoc status pills site-wide
2. Optional: DecisionBadge for DECISION\_STYLE tiers (A-Setup, Tradeable Watch, ...)

# Phase 1 — Quick Wins

Quelle: docs/frontend/PHASE1\_QUICK\_WINS.md

## Phase 1 — Quick Wins

**Status:** IN PROGRESS  
**Stand:** 16. August 2026  
**Branch:** feat/frontend-phase1-quickwins-2026-08-16  
**Bezug:** FRONTEND\_ROADMAP.md §4 Quick Wins, src/index.css , design-tokens.json

### Ziel

Sofort spürbare visuelle Ruhe und bessere Bedienbarkeit, ohne große Architektur-Umbauten.

| # | Quick Win                                 | Status                                    |
|---|-------------------------------------------|-------------------------------------------|
| 1 | Mehr Abstand zwischen Hauptkarten         | CSS-Utilities + Enterprise-Scorer Spacing |
| 2 | Score- / Matrix-Bereiche prominent        | teilweise (Enterprise-Layout)             |
| 3 | Einheitliche Status-Badges                | Phase 0 ( StatusBadge )                   |
| 4 | Mobile Chip- / Button-Größen (BFSG 44×44) | .ui-hit + Enterprise Filter/Timelineframe |
| 5 | Neural-Pulse abschwächen / Reduced-Motion | Keyframes + prefers-reduced-motion        |
| 6 | Klare primäre Aktion pro Viewport         | backlog                                   |

### CSS-Fundament ( src/index.css )

#### Motion (Phase 1)

- Neural-/Gold-Pulse: längere Dauer, geringere Scale ( $\pm 3\%$ ), weichere Glows
- Brand-Border-Animationen: langsamer und leiser
- @media (prefers-reduced-motion: reduce) : dekorative Animationen aus

#### Utilities

| Klasse    | Zweck                                         |
|-----------|-----------------------------------------------|
| .ui-panel | Standard-Glass-Card (blur + border + padding) |
| .ui-stack | Vertikaler Stack mit --ui-section-gap (2rem)  |
| .ui-hit   | min-height/width: 44px (BFSG / EN 301 549)    |

| Klasse                     | Zweck                                |
|----------------------------|--------------------------------------|
| <code>.neural-decor</code> | Opacity 0.4 für dekorative SVG-Layer |

## CSS-Variablen

```
--ui-section-gap: 2rem;  
--ui-card-gap: 1.5rem;  
--ui-hit-min: 44px;
```

## Call-Sites (dieser PR)

1. **Global** — `index.css` Motion + Utilities
2. **CryptoScoringEnterprise** — `space-y-8`, weichere Blur-Orbs, Filter-Chips und Timeframe-Buttons mit `min-h-11`

## Nächste Schritte (Phase 1 fortsetzen)

1. `.ui-stack` / größere Gaps in `Dashboard`, `AssetUniverseDashboard`, `MarketScreener`
2. Neural-SVG in `Dashboard` / `LandingPage` mit Klasse `neural-decor` markieren
3. Primäre CTA pro Viewport (eine dominante Aktion)
4. Einheitliche Button-/Chip-Primitive (analog StatusBadge)

Phase 1 gestartet 16.08.2026.

# Architecture

---

Quelle: `src/platform/Architecture/README.md`

## Architecture

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Architecture component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Compliance

---

Quelle: `src/platform/Compliance/README.md`

## Compliance

### Enterprise Component

Status: Implemented

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Backend des `SecurityComplianceAuditor` (ADR-0012):

- `scanners.ts`: 20+ Scanner-Regeln (Praefixe `SEC-`, `DAT-`, `GOV-`), die direkt gegen den echten Repository-Code pruefen (Dateiinhalte, Pfade, Konfigurationsmuster) - keine simulierten Ergebnisse. Jede Regel ist einem ISO/IEC-27001-Annex-A-Control zugeordnet.
  - `router.ts`: REST-API unter `/api/compliance/*` (`dashboard`, `run`, `certify`, `certificates`, `report`, `risk`), admin-geschuetzt ueber `checkAdminAccess()`.
  - `store.ts`: Persistenz von Scan-Ergebnissen, Remediation-Plaenen und Compliance-Zertifikaten in Supabase.
  - `types.ts`: gemeinsame Typen, auch client-seitig importiert (typreiner Import in `src/components/SecurityComplianceAuditor.tsx`, zur Kompilierzeit entfernt).
- 

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

ESS-0006 — Security & Compliance

---

### ADR References

ADR-0012 — Security & Compliance Auditor

---

### Dependencies

`server/db.ts`, `src/platform/Security/authMiddleware.ts`, `src/platform/Security/types.ts`.

---

## Events

Produziert und konsumiert aktuell keine Enterprise-Bus-Events (ESS-0013). Die zuvor im manifest.json genannten Events ( `ComplianceValidatedEvent` , `LayerViolationEvent` , `ContractViolationEvent` , `ArchitectureValidatedEvent` ) waren nie implementiert und wurden entfernt (ARCH-AUDIT-0002 J5).

---

## Notes

ARCH-AUDIT-0002 (J5, Kapitel 14.6): physisch aus `server/compliance/` hierher verschoben. Vorher deklarierte die ESS-Registry ( `implementedBy` von ESS-0006) diesen Ordner, obwohl er nur eine leere, vom Enterprise Bootstrapper generierte Huelle war und der tatsaechliche Code unter `server/compliance/` lag. Im Zuge der Verschiebung wurden vier Scanner-Regeln (SEC-04, SEC-05, DAT-03) korrigiert, die den frueheren Pfad `server/iam/authMiddleware.ts` als Literal-String pruefen - ohne diese Korrektur haetten sie nach der parallelen Verschiebung von `Security` dauerhaft falsche Befunde gemeldet, da `relPath` ein zur Laufzeit berechneter String ist und von `tsc` nicht geprueft wird.

# Contracts

---

Quelle: `src/platform/Contracts/README.md`

## Contracts

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Contracts component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# basisschicht

---

Quelle: `src/platform/Core/basisschicht.md`

# Core

---

Quelle: `src/platform/Core/README.md`

## Core

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Core component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Discovery

---

Quelle: `src/platform/Discovery/README.md`

## Discovery

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Discovery component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Documentation Governance Validator

Quelle: `src/platform/Documentary/Governance/README.md`

## Documentation Governance Validator

### Enterprise Component

Status: Unspecified (vollständig spezifiziert per ESS-0012, jedoch 0 Code-Dateien — siehe „Implementierungsstand“ unten)

Version: 1.0.0

Layer: Documentary

Owner: Documentary Engine

### Purpose

Der Documentation Governance Validator ist die Prüfinstanz der Documentary Engine.

Er überprüft sämtliche Dokumentation, Regeln, Metadaten und Beziehungen des Repositories automatisch auf Konsistenz, Vollständigkeit und Governance-Konformität.

Er erzeugt ausschließlich Befunde und Berichte.

Er trifft keine Entscheidungen, verändert keine Dokumentation und behebt keine Verstöße.

### Abgrenzung

| Instanz                                | Verantwortung                             |
|----------------------------------------|-------------------------------------------|
| Documentary Engine (ESS-0010)          | erzeugt Wissen und Dokumentation          |
| <b>Governance Validator (ESS-0012)</b> | <b>prüft Dokumentation und Governance</b> |
| Enterprise Traceability (ESS-0011)     | verknüpft Artefakte bidirektional         |
| Supervisor (ESS-0002)                  | bewertet Befunde und eskaliert            |
| Platform Director (ESS-0003)           | entscheidet                               |

Der Validator stellt fest. Der Supervisor bewertet. Der Platform Director entscheidet.

### Struktur

|                                                                 |                                                                                                                                |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Governance/<br>Contracts/<br>Validators/<br>Rules/<br>Services/ | komponentenspezifische Contracts<br>Einzelvalidatoren je Prüfbereich<br>Regelwerk, deklarativ<br>Ausführung und Orchestrierung |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|

|             |                             |
|-------------|-----------------------------|
| Reports/    | Berichtserzeugung           |
| Events/     | Event-Erzeugung und -Konsum |
| Models/     | Datenmodelle                |
| Interfaces/ | öffentliche Schnittstellen  |

Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß ESS-0001-CONTRACTS Chapter 3.

## Prüfbereiche

| Bereich         | Gegenstand                                                                                       |
|-----------------|--------------------------------------------------------------------------------------------------|
| ESS             | doppelte Regeln, fehlende Referenzen, ungültige Nummern, fehlende Cross-References, Widersprüche |
| ADR             | fehlende Referenzen, Widersprüche, veraltete ADRs, Breaking Changes ohne ADR                     |
| Repository      | README, CHANGELOG, manifest.json, component.yaml, Ownership                                      |
| Contracts       | Duplikate, Widersprüche, ungültige und nicht referenzierte Contracts                             |
| Dokumentation   | Vollständigkeit, Aktualität, Versionierung, Konsistenz, Struktur, Formatierung                   |
| Traceability    | fehlende Beziehungen entlang ESS → ADR → Code → Test → Doku → Version → Release                  |
| Knowledge Graph | verwaiste Knoten, doppelte, fehlende und inkonsistente Beziehungen                               |
| Digital Twin    | fehlende, neue, gelöschte und nicht synchronisierte Komponenten                                  |
| Versionierung   | Versionskonflikte über Dokument-, Komponenten-, Repository-, ESS- und ADR-Version                |

## ESS Reference

ESS-0012 — Documentation Governance

ESS-0012-CONTRACTS — Documentation Governance Contracts

ESS-0010 — Documentary Engine

ESS-0011 — Enterprise Traceability

ESS-0001-CONTRACTS — Master Enterprise Standard

## ADR References

ADR-0014 — Documentation Governance Validator

ADR-0013 — ESS Documentation Responsibility Consolidation

ADR-0010 — Enterprise Standard Extension

Dependencies

Zulässig: Core, Shared, Registry, Discovery, Knowledge

Unzulässig: VersionManager, Supervisor, PlatformDirector

Die Kommunikation nach oben erfolgt ausschließlich über Enterprise Events.

Events

Erzeugt

DocumentationValidatedEvent

GovernanceViolationDetectedEvent

DuplicateContractDetectedEvent

TraceabilityViolationEvent

DigitalTwinOutOfSyncEvent

RepositoryHealthUpdatedEvent

Konsumiert

RepositoryScannedEvent

KnowledgeUpdatedEvent

TwinSynchronizedEvent

ComponentRegisteredEvent

DocumentationGeneratedEvent

VersionChangedEvent

Implementierungsstand

Diese Komponente ist **spezifiziert, nicht implementiert**.

Die Implementierung setzt die Umsetzungsstufen 1 bis 4 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` voraus:

| Stufe | Voraussetzung | Zustand |
|-------|---------------|---------|
| 1     |               | offen   |

| Stufe | Voraussetzung                         | Zustand |
|-------|---------------------------------------|---------|
|       | JSON-Schemata, vollständige Metadaten |         |
| 2     | Core, Interfaces, Models, Registry    | offen   |
| 3     | Enterprise Event Bus                  | offen   |
| 4     | Validator-Basisklasse, Quality Gates  | offen   |

Ohne diese Grundlagen wäre der Validator ein isoliertes Skript ohne Event-Anbindung, ohne Registry-Eintrag und ohne Twin-Integration — und damit selbst ein Governance-Verstoß.

---

## Notes

Spezifikation vollständig in ESS-0012 und ESS-0012-CONTRACTS.

Skill-Beschreibung unter `.ai/skills/Documentation-Governance-Validator.md`.

# Documentary

---

Quelle: `src/platform/Documentary/README.md`

## Documentary

### Enterprise Component

Status: Partial Implementation

Version: 1.10.0

Component Version Authority: `manifest.json#version`

Document Schema Version Authority: `Versioning/DocumentaryVersion.ts#DOCUMENTARY_DOCUMENT_SCHEMA_VERSION`

Platform Version Authority: repository `package.json#version` via the existing Version Manager / release-governance contract

Owner: CAPITAL-AI

---

### Purpose

Documentary wird schrittweise zu einer ausführbaren Plattformkomponente ausgebaut. Implementiert sind der bilinguale Vocabulary-Layer, D0 Version Authority, D1 Code Discovery, Status-Event Drift Detection (Phase B, read-only), Status-Event Drift Updater (Phase C, header-only, Draft-PR / dryRun default), D3 Document Models/Provenance, D2 Core Engine, D5/E1/E4 Traceability/Event-Integration, D4 Review/Lifecycle Governance, D6 Generatoren/Renderer und D7 Knowledge Projection.

### Implemented Scope

- `Contracts/BilingualDocumentReference.ts`
- `Documentation/BilingualDocumentaryProjection.ts`
- `Versioning/DocumentaryVersion.ts`
- `Discovery/CodeEvidence.ts`
- `Discovery/StatusEventEvidence.ts`
- `Discovery/StatusEventDriftDetector.ts`
- `Discovery/StatusEventDriftUpdater.ts`
- `Models/DocumentaryDocument.ts`
- `Models/DocumentaryProvenance.ts`
- `Interfaces/IDocumentaryEngine.ts`
- `Engine/DocumentaryEngine.ts`

- `Events/DocumentaryEvents.ts`
- `Events/DocumentaryEventConsumer.ts`
- `Events/DocumentaryEventPublisher.ts`
- `Traceability/DocumentaryTraceability.ts`
- `Lifecycle/DocumentaryLifecycle.ts`
- `Generators/DocumentaryRenderer.ts`
- `Knowledge/DocumentaryKnowledgeProjection.ts`
- `Architecture/documentary-baseline.json`

## D7 Knowledge Integration

D7 erzeugt aus einem bereits governierten `DocumentaryDocument` eine deterministische Knowledge-Projektion mit Dokumentknoten, gerichteten Beziehungen, Source Commit, Dokument-Fingerprint, Concept-IDs, Traceability-IDs, Provenance-Referenzen und SHA-256-Prüfsumme.

Die Projektion ist ausschließlich ein Übergabevertrag an die in ESS-0009 spezifizierte zentrale Knowledge Engine. Sie persistiert keine Daten in `.ai/knowledge/`, startet keinen Knowledge Build und führt keine zweite Knowledge Registry ein. `src/platform/Knowledge` bleibt eine eigenständige Authority und ist bis zu einem separaten Implementierungsscope weiterhin specification-only.

## D6 Generators & Renderer

D6 rendert ausschließlich bereits erzeugte `DocumentaryDocument` -Modelle. Unterstützt werden die Dokumenttypen `architecture`, `component`, `api`, `runbook`, `release-evidence` und `handoff` mit dokumenttyp-spezifischen Abschnittsprofilen.

Als primäres Ausgabeformat wird deterministisches Markdown erzeugt. Für jedes Dokument können DE- und EN-Artefakte mit lokalisierten Metadatenüberschriften erstellt werden. Dateinamen, Provenance, Concept-IDs und Traceability-IDs werden stabil normalisiert und sortiert.

Der Renderer verändert weder den Lifecycle-Status noch den Dokument-Fingerprint. Er führt keine Freigabe, Persistenz, Source-Code-Mutation oder Event-Publikation durch.

## D4 Review & Lifecycle Governance

Der kontrollierte Lifecycle lautet `generated -> reviewed -> approved`. Nach Approval sind `approved -> superseded`, `approved -> archived` und `superseded -> archived` zulässig. Jeder Übergang benötigt eine explizite Actor-ID, passende Aktion, einen Zeitpunkt und mindestens eine Evidence-Referenz. Status-Sprünge, falsche Aktionen und evidence-freie Übergänge werden fail-closed blockiert.

Der separate Namespace `Governance/` bleibt dem in ESS-0012 spezifizierten Documentation Governance Validator vorbehalten. Dieser Validator ist weiterhin spezifiziert, aber nicht implementiert.

## D5 / E1 / E4 Traceability & Event Value Chain

`DocumentaryTraceabilityRecord` verbindet Dokumente mit `correlationId`, `causationId`, Dokument-Fingerprint, Source Commit, Concept IDs, Traceability IDs und Provenance-Referenzen. Documentary nutzt ausschließlich den bestehenden EventMesh.

## D2 Core Engine

`DocumentaryEngine` orchestriert die Erzeugung strukturierter `DocumentaryDocument`-Models und erzeugt ausschließlich den Lifecycle-Status `generated`.

## Version Model

- Component Version: `manifest.json#version`.
- Document Schema Version: `DOCUMENTARY_DOCUMENT_SCHEMA_VERSION`.
- Platform Version: zentrale Version-Manager-/ `package.json`-Authority.

## Implementation Baseline

Aktuell implementiert: `Contracts`, `Discovery`, `Documentation`, `Engine`, `Events`, `Generators`, `Interfaces`, `Knowledge`, `Lifecycle`, `Models`, `Traceability`, `Versioning`.

Weiterhin geplant: `Governance` (Documentation Governance Validator), `Mermaid`, `Migration`, `Plugins` sowie weitere Architecture-Runtime-Funktionen.

## Boundaries

Keine autonome Approval-Transition, keine Source-Code-Mutation, keine zweite Event- oder Knowledge-Infrastruktur. IAM-Step-Up/Persistenz, Governance Validator Runtime und der zentrale ESS-0009 KnowledgeBuilder bleiben separate Scopes. Phase-C Status-Header-Updates erfolgen nur header-only, allowlisted, dryRun-default und über Draft-PR.

## ESS / ADR

- ESS-0009 — Enterprise Knowledge Platform
- ESS-0010 — Documentary Engine
- ESS-0011 — Enterprise Traceability
- ESS-0012 — Documentation Governance
- ESS-0017 / ESS-0017-CONTRACTS — Vocabulary Governance
- ADR-0046 — Vocabulary Governance Authority and Namespace

# Changelog — EventMesh

---

Quelle: `src/platform/EventMesh/CHANGELOG.md`

## Changelog — EventMesh

Alle Änderungen an dieser Komponente werden hier dokumentiert.

Format gemäß ESS-0001-CONTRACTS Chapter 7, CHANGELOG Contract: Version, Datum, Beschreibung, Breaking Changes, Autor.

Die Versionierung folgt ESS-0001-CONTRACTS Chapter 9 (Semantic Versioning).

---

### [1.1.0] — 2026-07-31

#### Hinzugefügt

- Ausführbare Implementierung von Stufe 1 bis 4 (ADR-0018, Folgeentscheidung 1) — erste Komponente im gesamten `src/platform/`-Baum mit Code:
- `Contracts/` — `EventVersion`, `EventMetadata`, `EventPayload`, `EventSchema`, `EventContract`
- `Interfaces/` — `IEventBus`, `IEventPublisher`, `IEventSubscriber`, `IEventRouter`, `IEventRegistry`
- `Models/` — `EventDeliveryRecord`, `ManifestEventsDeclaration`
- `Registry/` — `EventRegistry`, `ProducerRegistry`, `ConsumerRegistry`, `EventCatalog`
- `Core/` — `EventBus`, `EventDispatcher`, `EventPublisher`, `EventSubscriber`, `EventRouter`
- `Validators/` — `EventContractValidator`, `EventSchemaValidator`, `EventVersionValidator`, `EventCompatibilityValidator`
- `Reports/` — `EventFlowReport`, `EventCoverageReport`, `EventHealthReport`, `EventDependencyReport`
- `Policies/RoutingPolicy.ts` — Retry-Policies je Event-Kategorie
- `Discovery/ManifestDiscovery.ts` — liest `manifest.json` aller Komponenten zur Laufzeit
- `Services/EventMeshService.ts` — Bootstrap (Katalog-Seed + Discovery)
- `Events/StandardEventCatalog.ts` — Ausgangsbestand von 36 Events (inkl. `SystemAuditEvent`)
- `Tests/eventBus.test.ts` — sieben Vertrags- und Kompatibilitätstests, `npm run test`-ausführbar
- `Services/SystemAuditBridge.ts` (Folgeentscheidung 3): additive Brücke zwischen dem bestehenden, produktiven Audit-Log (`server/systemEvents.ts`) und der Enterprise Event Mesh. `logSystemEvent()` veröffentlicht zusätzlich ein `SystemAuditEvent` in einem eigenen `try/catch` — ein Fehler dort kann niemals den bestehenden Datei-Schreibvorgang oder das SSE-Broadcast an das Admin-Portal gefährden. Per Rauchtest verifiziert: bestehendes Verhalten unverändert, Event wird zusätzlich zugestellt.

- Manifest-Nachpflege (Folgeentscheidung 2/4): `events`-Felder in 12 zuvor leeren Komponenten-Manifesten befüllt, 7 neue Events registriert.

## Befund während der Implementierung

`Discovery/ManifestDiscovery.ts` fand zur Laufzeit **23 Komponenten** unter `src/platform/` — mehr als die neun in der Vorab-Analyse identifizierten. Zusätzlich gefunden: `Core`, `Contracts`, `Events`, `Interfaces`, `Models`, `Registry`, `Shared`, `Telemetry`, `Validators`, `Plugins`, `Generators` — bislang nicht einzeln geprüfte Querschnittsverzeichnisse. Bestätigt den Wert automatisierter Discovery gegenüber rein manueller Analyse.

Ein echter Fehler wurde durch die Testsuite gefunden und korrigiert: `EventRegistry.registerProducer()` / `registerConsumer()` legten keinen Katalogeintrag an, wenn das Event zuvor nicht über `registerEvent()` geseedet worden war — im Widerspruch zu ESS-0013-CONTRACTS Abschnitt 3 ("Jedes Event wird bei Veröffentlichung automatisch registriert"). Behoben durch automatisches Anlegen eines minimalen Katalogeintrags.

## Breaking Changes

Keine.

## Autor

Platform Director

---

## [1.0.0] — 2026-07-31

### Hinzugefügt

- Komponentenstruktur mit zwölf Unterverzeichnissen gemäß ADR-0018 ( `Contracts`, `Core`, `Discovery`, `Events`, `Interfaces`, `Models`, `Policies`, `Registry`, `Reports`, `Services`, `Tests`, `Validators` )
- `manifest.json` mit vollständigem Metadatensatz nach Chapter 7
- `component.yaml` als menschen- und KI-lesbarer Komponentendeskriptor
- `README.md` mit Verantwortungsabgrenzung, umgekehrter Abhängigkeitsregel und Standard-Event-Katalog-Verweis
- Diese Changelog-Datei

### Referenzen

- ESS-0001-CONTRACTS Chapter 8 — Enterprise Event & Messaging Contracts (Regelwerk, unverändert)
- ESS-0013 — Enterprise Event Mesh (Spezifikation)
- ESS-0013-CONTRACTS — Event-Katalog, Kompatibilitäts- und Policy-Regeln
- ADR-0018 — Einführung der Enterprise Event Mesh als Plattformmodul

## Breaking Changes

Keine. Die Komponente wird ergänzend eingeführt und verändert keine bestehende Komponente, keinen bestehenden Contract und keinen produktiven Code.

## Implementierungsstand

Spezifiziert, nicht implementiert. Es existiert kein ausführbarer Code.

Die Umsetzung entspricht Stufe 3 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` (Enterprise Event Bus) und setzt Stufe 1 und 2 (Schema/Metadata Foundation, Core/Contracts) voraus.

## Autor

Platform Director

---

## Hinweis zur Erstfassung

Diese Komponente ist die **zweite** im Repository mit vollständigem Metadatensatz ( `component.yaml` , `CHANGELOG.md` ), nach `src/platform/Traceability/` (ADR-0015), und folgt demselben Muster.

# EventMesh

Quelle: `src/platform/EventMesh/README.md`

## EventMesh

### Enterprise Component

Status: Implemented / Operational

Version: 1.2.0

Layer: Querschnittsmodul mit erweitertem Zugriff

Owner: Platform Director

### Purpose

Die Enterprise Event Mesh (EEM) ist die zentrale Kommunikationsschicht der CAPITAL-AI Plattform. Sie vermittelt sämtliche Kommunikation zwischen Plattformkomponenten als standardisierte, versionierte Enterprise Events.

Sie erzeugt keine fachlichen Events selbst. Sie registriert, validiert, routet und protokolliert Events, die von den Fachkomponenten veröffentlicht werden.

Direkte Engine-zu-Engine-Abhängigkeiten sind langfristig durch registrierte Enterprise Events zu ersetzen.

### Verhältnis zur Spezifikation

Diese Komponente **implementiert** ESS-0013 sowie den bereits bestehenden, übergeordneten Rahmen aus **ESS-0001-CONTRACTS Chapter 8** (Enterprise Event & Messaging Contracts).

| Dokument                     | Rolle                                                                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| ESS-0001-CONTRACTS Chapter 8 | globales Regelwerk — Event-Prinzip, Namensregel, Contract-Pflichtfelder, Producer-/Consumer-Regeln, Routing-Prinzip, Validation-Kriterien |
| ESS-0013                     | Spezifikation der EventMesh- <b>Komponente</b> — Core-Klassen, Registry-Mechanik, Validator-Kette, Report-Typen                           |
| ESS-0013-CONTRACTS           | Regelwerk — Event-Katalog, Kompatibilitätsregeln, Policy-Regeln                                                                           |
| <b>diese Komponente</b>      | <b>Implementierungsort</b>                                                                                                                |

Chapter 8 wird hier **nicht wiederholt**. Bei Abweichungen gilt Chapter 8, danach ESS-0013.

## Struktur

|             |                                                                                                         |
|-------------|---------------------------------------------------------------------------------------------------------|
| EventMesh/  |                                                                                                         |
| Contracts/  | Event-Contract-Definitionen (EventContract, EventPayload, EventMetadata, EventVersion, EventSchema)     |
| Core/       | EventBus, EventDispatcher, EventPublisher, EventSubscriber, EventRouter                                 |
| Discovery/  | automatische Erkennung möglicher Producer und Consumer                                                  |
| Events/     | konkrete Enterprise-Event-Klassen je Kategorie                                                          |
| Interfaces/ | öffentliche Schnittstellen (IEventBus, IEventPublisher, IEventSubscriber, IEventRouter, IEventRegistry) |
| Models/     | Datenmodelle für Registry-Einträge, Katalogeinträge, Abonnements                                        |
| Policies/   | Routing-Policies, Retry-Policies, Kompatibilitätsolicies                                                |
| Registry/   | EventRegistry, ProducerRegistry, ConsumerRegistry, EventCatalog                                         |
| Reports/    | EventFlowReport, EventCoverageReport, EventHealthReport, EventDependencyReport                          |
| Services/   | Ausführung und Orchestrierung des Routings                                                              |
| Tests/      | Vertrags- und Kompatibilitätstests                                                                      |

Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß ESS-0001-CONTRACTS Chapter 3.

## Abhängigkeiten

**Zulässig:** Core, Shared, Registry, Discovery

**Unzulässig:** Supervisor, PlatformDirector, Documentary, Traceability, Knowledge, Compliance, VersionManager, Security, Release, Quality — **jede** fachliche Komponente kommuniziert mit der EventMesh ausschließlich über die in `Interfaces/` definierten Schnittstellen, niemals über einen direkten Modul-Import.

Diese Regel ist die Umkehrung der üblichen Abhängigkeitsrichtung: Während andere Querschnittsmodule (z. B. Traceability, Release) begrenzten Zugriff auf mehrere Komponenten benötigen, darf **keine** Fachkomponente einen direkten Zugriff auf die EventMesh-Implementierung besitzen — nur auf ihre öffentlichen Interfaces. Andernfalls entstünde exakt die Kopplung, die die EventMesh auflösen soll.

Die Einordnung als Querschnittsmodul mit erweitertem Zugriff folgt derselben Regelung aus ESS-0001-CONTRACTS Chapter 16 (Cross Cutting Modules), die bereits für `Release` und `Traceability` angewendet wurde (ADR-0015).

## Rollentrennung

| Instanz                                                                                                                               | Rolle                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>EventMesh</b>                                                                                                                      | <b>registriert, validiert, routet, protokolliert Events</b> |
| Fachkomponenten (Documentary, Traceability, Knowledge, Compliance, Version Manager, Supervisor, Platform Director, Security, Release) | erzeugen und konsumieren fachliche Events                   |

| Instanz              | Rolle                                             |
|----------------------|---------------------------------------------------|
| Governance Validator | prüft Event-Contracts auf Dokumentationsregeln    |
| Supervisor           | überwacht Event-Flüsse, -Ausfälle, -Muster        |
| Platform Director    | entscheidet über neue Event-Typen und -Kategorien |

Die EventMesh erzeugt keine fachlichen Entscheidungen. Sie vermittelt.

## Event-Katalog (Referenzstand)

Der vollständige, versionierte Katalog liegt in `Registry/ (EventCatalog)` und wird automatisch aus den Producer-Deklarationen der Fachkomponenten aufgebaut. Der Ausgangsbestand von 15 Standardereignissen sowie deren korrekte kanonische Namen und Quellkapitel sind in ESS-0013 Abschnitt Standard Event Catalog dokumentiert — dort auch die Korrektur dreier in der auslösenden Anforderung abweichend benannter Events (`TraceabilityUpdatedEvent` → `TraceabilityBuildCompletedEvent`, `KnowledgeGraphUpdatedEvent` → `KnowledgeUpdatedEvent`, `DigitalTwinUpdatedEvent` → `TwinSynchronizedEvent`), um Duplikate zu bereits kanonischen Event-Namen aus Chapter 8/15/18 und ESS-0011 zu vermeiden.

## E2 — Idempotency, Ordering & Replay

`Policies/EventReplayGuard.ts` verwendet `eventId` als Idempotency Key. Ein deliberate Replay derselben Event-ID wird als `duplicate` klassifiziert. Ordering wird ausschließlich innerhalb derselben `correlationId` bewertet; ein älter eintreffendes Event wird dort als `stale` klassifiziert, ohne unabhängige Correlations zu blockieren.

Die Guard-Schicht ist fail-closed bei fehlender Event-ID, Correlation-ID oder ungültigem Timestamp. Sie bleibt in-memory und pro Prozess. Durable Multi-Instance-Deduplication bleibt ein separater Infrastruktur-Scope.

## E5 — Reliability & Observability Evidence

`Reports/EventReliabilityReport.ts` leitet aus dem bestehenden Delivery Log deterministisch Delivery-/Failure-/No-Consumer-Zähler, Failure Rate, fehlgeschlagene Event-Typen und Consumer, strukturierte Failure Evidence sowie Poison-Event-Kandidaten ab. Die Auswertung ist read-only und erzeugt keine zweite Observability-Pipeline oder autonome Retry-Entscheidung.

## ESS Reference

ESS-0013 — Enterprise Event Mesh

ESS-0013-CONTRACTS — Enterprise Event Mesh Contracts

ESS-0001-CONTRACTS Chapter 8 — Enterprise Event & Messaging Contracts (Regelwerk, unverändert)

ESS-0011 — Enterprise Traceability (Event-Achse der ETM)

ESS-0010 — Documentary Engine

## ADR References

ADR-0018 — Einführung der Enterprise Event Mesh als Plattformmodul

ADR-0015 — Enterprise Traceability Component (Vorbild für Querschnittsmodul-Einordnung)

ADR-0014 — Documentation Governance Validator

ADR-0013 — ESS Documentation Responsibility Consolidation

## Implementierungsstand

Diese Komponente ist **implementiert (Stufe 1-4)** — als erste Komponente im gesamten `src/platform/`-Baum mit ausführbarem Code (ADR-0018, Folgeentscheidung 1).

| Stufe | Voraussetzung                                            | Zustand                                  |
|-------|----------------------------------------------------------|------------------------------------------|
| 1     | JSON-Schemata für Event Contract, vollständige Metadaten | erledigt — <code>Contracts/</code>       |
| 2     | Core, Interfaces, Models, Registry                       | erledigt                                 |
| 3     | Enterprise Event Bus (Routing, Zustellung)               | erledigt — <code>Core/EventBus.ts</code> |
| 4     | Validator-Kette ( <code>Validators/</code> )             | erledigt — vier Validatoren              |

In-Memory, ohne externe Abhängigkeiten. E2/E5 ergänzen Replay-/Ordering- und Reliability-Evidence, führen aber bewusst keine persistente Queue oder zweite Event-Infrastruktur ein.

`Discovery/ManifestDiscovery.ts` liest `manifest.json` aller `src/platform/`-Komponenten zur Laufzeit und bildet Producer-/Consumer-Evidence aus dem realen Repository-Bestand.

`server/systemEvents.ts` (Audit-Log) und die Frontend- `CustomEvent` -Nutzung wurden additiv erweitert (ADR-0018, Folgeentscheidung 3); der bestehende Mechanismus bleibt erhalten.

## Notes

Vollständige Analyse und Begründung: `docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md`, `docs/architecture/ENTERPRISE_EVENT_MESH_READINESS_REPORT.md` und `docs/architecture/EVENTMESH_E2_E5_REPLAY_RELIABILITY.md`.

# Events

---

Quelle: `src/platform/Events/README.md`

## Events

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Events component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutaeuschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Generators

---

Quelle: `src/platform/Generators/README.md`

## Generators

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Generators component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Interfaces

---

Quelle: `src/platform/Interfaces/README.md`

## Interfaces

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Interfaces component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Knowledge

---

Quelle: `src/platform/Knowledge/README.md`

## Knowledge

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Knowledge component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Models

---

Quelle: `src/platform/Models/README.md`

## Models

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Models component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutaeuschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# PlatformDirector

---

Quelle: `src/platform/PlatformDirector/README.md`

## PlatformDirector

### Enterprise Component

Status: Development

Version: 1.2.0

Owner: CAPITAL-AI

---

### Purpose

PlatformDirector enthaelt die Contracts fuer geschuetzte Plattformscheidungen, die EventMesh-Bridge fuer bereits genehmigte `PlatformDecisionRecord`s und mit E6 eine explizite fail-closed Protected Decision Boundary. Die eigentliche Entscheidungsfindung und Persistenz bleibt weiterhin ausserhalb dieses Implementierungsschritts.

### Implementierter Scope

- `Contracts/PlatformDecision.ts` : Architecture-, Governance-, Release-, Exception-, Risk-, Priority- und Emergency-Decision Contracts.
- `Events/publishPlatformDecision.ts` : propagiert ausschliesslich bereits `APPROVED` Entscheidungen als kanonisches `PlatformDecisionEvent`.
- `Policies/ProtectedDecisionBoundary.ts` : validiert E6 Governance Boundaries vor der Propagation.

### E6 — Governance Boundaries

Eine geschuetzte Entscheidung darf die Boundary nur passieren, wenn sie explizit `APPROVED` ist, vom `Platform Director` entschieden wurde, eine Correlation-ID sowie Decision-Basis-Evidence besitzt und keine geblockte Supervisor-Evidence enthaelt.

Fuer `Release Decision` gelten zusaetzlich verpflichtend:

- Release-Candidate-Evidence;
- nicht-leerer Rollback-Plan;
- `qualityGate: PASS`;
- `securityGate: PASS`;
- `complianceGate: PASS`;
- `versionGate: PASS`.

`FAIL` oder `UNAVAILABLE` blockieren den Release-Pfad. Die Policy genehmigt nichts autonom und ersetzt weder Release Center, Version Manager, Quality, Security, Compliance noch EventMesh.

## Schutzgrenzen

Keine autonome Approval-Entscheidung, keine zweite Governance Engine, keine zweite EventMesh oder Registry und keine Persistenzmutation. Geschuetzte Entscheidungen bleiben Human-/Platform-Director-gesteuert.

## ESS Reference

- ESS-0001
- ESS-0001-CONTRACTS
- ESS-0003 — Platform Director
- ESS-0013 — Enterprise Event Mesh
- ESS-0017 — Vocabulary Governance

## ADR References

- ADR-0018 — Enterprise Event Mesh

## Notes

Die Komponente bleibt `development`, weil Entscheidungsfindung und persistente Decision Records noch nicht vollstaendig als PlatformDirector-Runtime implementiert sind. E6 ergaenzt ausschliesslich eine testbare Governance-Grenze fuer bereits vorhandene Decision-Evidence.

# Plugins

---

Quelle: `src/platform/Plugins/README.md`

## Plugins

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Plugins component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Quality

---

Quelle: `src/platform/Quality/README.md`

## Quality

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Quality component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Registry

---

Quelle: `src/platform/Registry/README.md`

## Registry

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Registry component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Release

---

Quelle: `src/platform/Release/README.md`

## Release

### Enterprise Component

**Status:** Development

**Component Version:** 1.1.0

**Owner:** Platform Director

**Governance:** ADR-0030 / GOV-VER-002

**Runtime Contract:** `release-version-gate/1.0.0`

---

### Purpose

The Release component governs CAPITAL-AI platform-version advancement and release-candidate evidence. It does not decide that a feature is ready by itself and it never creates a final Git tag.

`package.json#version` remains the single source of truth for the currently released platform version. The Release component reads that value at execution time; this document intentionally does not pin a separate current platform version.

A version changes only at the dedicated Release Version Gate defined by ADR-0030.

---

### Controlled Command

```
npm run release:version -- \
  --target=0.7.0 \
  --classification=MINOR \
  --work-packages=PR-54,PR-56 \
  --adrs=ADR-0032,ADR-0033 \
  --migrations=none \
  --risks="provider availability remains plan-dependent" \
  --rollback-boundary="Rollback to the exact prior accepted artifact" \
  --acceptance="production smoke tests,provider health evidence"
```

The command is a **dry run by default**. It validates the requested transition and all mandatory release metadata without changing files.

Only an intentional execution with `--apply` may mutate governed version declarations:

```
npm run release:version -- <same arguments> --apply
```

---

## Fail-Closed Rules

The gate rejects a request when, among other conditions:

- the target is not strict `MAJOR.MINOR.PATCH` SemVer;
- the target is not greater than the current release;
- a PATCH skips more than one patch level;
- a MINOR skips a minor line or does not reset patch to zero;
- a MAJOR is not the next major `.0.0` version;
- PATCH/MINOR/MAJOR classification does not match the requested transition;
- a MINOR/MAJOR has no ADR traceability;
- a MAJOR has no dedicated GA ADR;
- work-package scope, rollback boundary, known risks or production-acceptance requirements are missing;
- `package.json` and the root `package-lock.json` version are already inconsistent;
- a governed mirror declaration cannot be synchronized;
- any mandatory post-update release check fails.

There is no best-effort or partial version bump. If an apply-stage gate fails, the version files are restored to their pre-gate content.

---

## Governed Version Declarations

The gate synchronizes:

1. `package.json`
2. root `package-lock.json#version`
3. `package-lock.json#packages[""]#version`
4. `metadata.json`
5. `README.md`
6. `AGENTS.md`
7. `docs/code-quality/CODE_QUALITY_STANDARDS.md`
8. `docs/ceo/EXECUTIVE_SUMMARY.md`
9. `docs/API.md`
10. `index.html`

The regression test `tests/unit/platformVersionConsistency.test.ts` independently verifies the source-of-truth relationship.

---

## Mandatory Apply Gates

After synchronization, the release command executes:

```
npm run lint
npx vitest run tests/unit/platformVersionConsistency.test.ts
npm run build
npm run predeploy:check
```

Any non-zero result aborts the release-version operation and restores the governed version files.

The normal repository CI still runs independently after the change is committed; the local gate is not a substitute for GitHub CI.

---

## Release Candidate Evidence

Only after all apply-stage gates pass, the command creates:

```
docs/releases/candidates/RELEASE_CANDIDATE_<target-version>.md
```

The candidate record contains:

- source and target version;
- PATCH/MINOR/MAJOR classification;
- work-package / PR scope;
- ADR scope;
- migration scope;
- known risks;
- rollback boundary;
- production-acceptance requirements;
- version-gate evidence;
- explicit status `VERSIONED_RC_PENDING_PRODUCTION_ACCEPTANCE` ;
- explicit `NOT_CREATED` final-tag state.

The exact candidate commit SHA is resolved by GitHub CI / the production acceptance record. The candidate evidence file cannot safely embed the SHA of the same commit that contains the file, because doing so would create an impossible self-referential commit hash.

---

## Production Acceptance and Final Tag

This component deliberately **does not** implement `git tag`.

The lifecycle boundary is:

```
Development
-> Integration / Scope Freeze
-> Release Version Gate
-> Versioned Release Candidate
-> Production Deployment
```

```
-> Production Acceptance  
-> immutable tag vMAJOR.MINOR.PATCH  
-> Traceability Closure
```

A failed candidate must never be tagged as the released platform version. A final tag may be created only after the exact deployed commit has a closed production acceptance record.

Historical final tags must never be moved to different commits.

---

## Tests

- `tests/unit/releaseVersionGate.test.ts`
  - exact next PATCH/MINOR/MAJOR transitions;
  - classification mismatch rejection;
  - GA ADR requirement;
  - package-lock drift rejection;
  - synchronized update and rollback;
  - mandatory ADR / acceptance traceability.
  - `tests/unit/platformVersionConsistency.test.ts`
  - strict SemVer source of truth;
  - package-lock root consistency;
  - metadata consistency;
  - governed declaration consistency.
- 

## Enterprise References

- `docs/adr/resolved/ADR-0030-platform-version-release-lifecycle.md`
- `ESS-0001`
- `ESS-0001-CONTRACTS`
- `ESS-0007` — Enterprise Release Center
- `src/platform/Traceability`

The implementation does not change the application version by itself. A version increase remains a separate, explicit release operation under ADR-0030.

# Security

---

Quelle: `src/platform/Security/README.md`

## Security

### Enterprise Component

Status: Implemented

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

IAM-Kern der Plattform:

- `authMiddleware.ts` : `resolveVerifiedIdentity()` (E-Mail-basierte Identitätsverifikation, konsistent mit dem Subscription-Modell), `checkAdminAccess()` (rollenbasierte Autorisierung, fail-closed - jeder Fehlerpfad liefert `authorized: false`), `requireStepUp()` (frischer TOTP-Nachweis fuer kritische Owner-Aktionen, ADR-0003.5), `logIamEvent()`.
  - `rateLimiter.ts` : In-Memory-Zaehler fuer Admin-/Auth-kritische Endpunkte (Prozess-lokal, siehe Einschraenkung im Dateikopf zu horizontaler Skalierung).
  - `secretCrypto.ts` : AES-256-GCM-Verschluesselung ruhender Secrets (TOTP-Seeds), Opaque-Token Hashing.
  - `totp.ts` : RFC-6238-TOTP-Implementierung ohne externe Abhaengigkeit.
- 

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

ESS-0006 — Security & Compliance

---

### ADR References

ADR-0003.5 — Step-Up-Authentication

---

### Dependencies

`server/db.ts` , `server/env.ts` , `server/logger.ts` .

---

## Events

Produziert und konsumiert aktuell keine Enterprise-Bus-Events (ESS-0013).

Sicherheitsereignisse werden ueber `logSystemEvent()` (`server/systemEvents.ts`) protokolliert. Die zuvor im `manifest.json` genannten Events (`SecurityViolationDetectedEvent`, `ComplianceValidatedEvent`) waren nie implementiert und wurden entfernt (ARCH-AUDIT-0002 J5).

---

## Notes

ARCH-AUDIT-0002 (J5, Kapitel 14.6): physisch aus `server/iam/` hierher verschoben. Vorher deklarierte die ESS-Registry ( `implementedBy` von ESS-0006) diesen Ordner, obwohl er nur eine leere, vom Enterprise Bootstrapper generierte Huelle war und der tatsaechliche Code unter `server/iam/` lag.

# Shared

---

Quelle: `src/platform/Shared/README.md`

## Shared

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Shared component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutauschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# Supervisor

---

Quelle: `src/platform/Supervisor/README.md`

## Supervisor

### Enterprise Component

Status: Implemented (extended 2026-08-16)

Version: 1.2.0

Owner: CAPITAL-AI

---

### Purpose

`supervisor.ts` is the real Supervisor component.

### Implemented:

- **Task Routing / Tool Selection** — `routeTask()`
- **Execution Control / Retry / Recovery / Self-Healing** — `executeSupervised()`
- **Approved write path** — `executeApprovedSupervisedAction()` (Policy → Approval → Apply → Audit)
- **Agent Provider Chain Observation** (2026-08-16) — ChatGPT, Claude, Grok via `observeAgentProviderChain()`; Google AI Studio/NotebookLM/Gemini = RETIRED
- **Findings** — lightweight findings from failed supervised executions and provider inventory (ESS-0002 spirit; Supervisor does not decide)

### Not implemented:

- Multi-engine Conflict Resolution (one authoritative engine per asset class)
  - Full ESS-0002 Digital Twin / complete finding lifecycle persistence
- 

### ESS / ADR

ESS-0001, ESS-0001-CONTRACTS, ESS-0002; ADR-0018, ADR-0051, ADR-0062

# Telemetry

---

Quelle: `src/platform/Telemetry/README.md`

## Telemetry

### Enterprise Component

Status: Development

Version: 1.1.0

Owner: CAPITAL-AI

### Zweck

Telemetry ist die vendor-neutrale Operational-Evidence-Schicht von CAPITAL-AI. Sie standardisiert Logs, Metriken und Trace-Kontext entlang der FinTech-Wertschöpfungskette, ohne Security-/Compliance-Audit-Evidence zu ersetzen.

Die Komponente baut auf dem bestehenden `server/logger.ts` auf. Es wird bewusst kein zweites Logger-System eingeführt.

### Architekturgrenzen

- Operational Telemetry: Logs, Metriken, Trace-/Correlation-Kontext, Latenzen und technische Outcomes.
- Audit Evidence: unveränderbare IAM-, Break-Glass-, Billing-/Compliance- und Governance-Evidence bleibt in den zuständigen Authorities.
- Keine Secrets, Tokens, Passwörter oder direkte PII in Operational Telemetry.
- Kein Request-Body und kein Query-String in der generischen Request-Telemetry.
- OpenTelemetry ist das vorgesehene spätere Export-/Tracing-Protokoll; O1 fügt noch keinen externen Vendor oder Collector hinzu.

### FinTech-Wertschöpfungskette

Der Contract kennt folgende kanonische Stufen:

1. `request-intake`
2. `identity-access`
3. `entitlement-usage`
4. `orchestration`
5. `market-data-provider`
6. `data-validation`
7. `scoring-analysis`
8. `explainability`

9. `billing`
10. `output-delivery`
11. `deployment-runtime`

Damit können spätere Metriken und Traces vom eingehenden Request über Provider und Scoring bis zur Ausgabe und Produktionsversion korreliert werden.

## O1 - implementiert

- typisierter `TelemetryRecord` mit Schema-Version;
- kanonische Event-Namen und Outcomes;
- Request-/Trace-/Span-Kontext im Contract;
- zentrale Secret-/PII-Redaction;
- bestehender JSON-Logger um Service, Environment, Version und Commit erweitert;
- validierte `x-request-id`-Übernahme mit sicherem UUID-Fallback;
- generische Request-Abschlussmetrik mit Status und Dauer;
- `/healthz` wird aus der regulären Request-Telemetrie ausgeschlossen;
- Contract-/Redaction-Tests.

## Bestehende Integrationspunkte

Die nächsten Phasen instrumentieren vorhandene Authorities statt parallele Systeme aufzubauen:

- IAM / Step-Up: `src/platform/Security`, `server/stepUp.ts`;
- Orchestrierung: `src/lib/requestOrchestrator.ts`, `server/orchestrator.ts`;
- Provider Routing: `src/services/marketDataProviderRouter.ts`;
- Screening SLA: `src/services/screeningSla.ts`;
- Compliance Evidence: `src/platform/Compliance/screeningGovernanceEvidence.ts`;
- Supervisor: `src/platform/Supervisor/supervisor.ts`;
- EventMesh: `src/platform/EventMesh`;
- Deployment Evidence: `server/deploymentIdentity.ts` und GitHub→Render Deploy Gate.

## ESS / Architecture References

- ESS-0001 / ESS-0001-CONTRACTS — Enterprise Component Contract
- ESS-0005 — Quality Center
- ESS-0006 — Security & Compliance
- ESS-0011 — Enterprise Traceability
- ESS-0013 — Enterprise Event Mesh
- ADR-0056 — Observability & Telemetry Baseline

## **Folgephasen**

O2: strukturierte Logger-Migration und Error Taxonomy.

O3: technische und fachliche Metriken.

O4: OpenTelemetry / Distributed Tracing.

O5: FinTech Business Telemetry.

O6: SLI/SLO und Alerting.

O7: Production Evidence.

O8: Continuous Telemetry Governance.

# Changelog — Traceability

---

Quelle: `src/platform/Traceability/CHANGELOG.md`

## Changelog — Traceability

Alle Änderungen an dieser Komponente werden hier dokumentiert.

Format gemäß ESS-0001-CONTRACTS Chapter 7, CHANGELOG Contract: Version, Datum, Beschreibung, Breaking Changes, Autor.

Die Versionierung folgt ESS-0001-CONTRACTS Chapter 9 (Semantic Versioning).

---

### [1.0.0] — 2026-07-31

#### Hinzugefügt

- Komponentenstruktur mit elf Unterverzeichnissen gemäß ADR-0015 ( `Contracts` , `Core` , `Discovery` , `Events` , `Interfaces` , `Models` , `Registry` , `Reports` , `Services` , `Validators` , `Versioning` )
- `manifest.json` mit vollständigem Metadatensatz nach Chapter 7
- `component.yaml` als menschen- und KI-lesbarer Komponentendeskriptor
- `README.md` mit Verantwortungsabgrenzung und Abhängigkeitsregeln
- Diese Changelog-Datei

#### Referenzen

- ESS-0011 — Enterprise Traceability (Spezifikation)
- ESS-0011-CONTRACTS — Link Contract, Coverage Contracts, Orphan Contracts
- ADR-0015 — Einführung der Traceability-Komponente

#### Breaking Changes

Keine. Die Komponente wird ergänzend eingeführt und verändert keine bestehende Komponente.

#### Implementierungsstand

Spezifiziert, nicht implementiert. Es existiert kein ausführbarer Code.

Die Umsetzung setzt die Stufen 1 bis 4 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` voraus — insbesondere den Enterprise Event Bus und die Validator-Basisklasse.

#### Autor

Platform Director

---

## [1.1.0] — 2026-08-02

### Hinzugefügt

- `Models/traceabilityModels.ts` — Datenmodelle (`EssArtifact`, `AdrArtifact`, `ComponentArtifact`, `TestArtifact`, `TraceabilityLink`, `TraceabilityMatrix`, `CoverageReport`, `OrphanFinding`)
- `Interfaces/index.ts` — `ITraceabilityBuilder`, `ICoverageAnalyzer`, `IOrphanDetector`, `ITraceabilityReporter`, `ITraceabilityLink`
- `Discovery/artifactDiscovery.ts` — liest ESS-Registry, ADR-Historie, alle Plattform-Manifeste (rekursiv) und alle Testdateien unter `tests/`
- `Core/traceabilityBuilder.ts` — baut die Achsen `ESS↔Component` und `Component↔Test`
- `Core/coverageAnalyzer.ts`, `Core/orphanDetector.ts`
- `Registry/traceabilityRegistry.ts` — Prozessregistrierung des zuletzt gebauten Standes
- `Reports/traceabilityReporter.ts` — schreibt `matrix.json` / `coverage.json` / `orphans.json` unter `.ai/knowledge/traceability/` und `docs/traceability/COVERAGE_REPORT.md`
- `Services/runTraceability.ts` — CLI, `npm run traceability:build`; führt zusätzlich das `tests`-Feld in allen 25 Plattform-Manifesten anhand real gefundener Testdateien nach

### Geändert

- `manifest.json`: `status` von `specified` auf `development`; Klarstellung, dass die deklarierten Events (noch) nicht ausgelöst/konsumiert werden und die deklarierten Abhängigkeiten (Core, Shared, Registry, Knowledge, Discovery) mangels eigenen Codes nicht importiert werden
- `README.md`: Implementierungsstand aktualisiert (Stufe 1-2 umgesetzt, 3-4 offen); die zuvor genannte KG-/Digital-Twin-Blockade als für diesen Ausbau nicht zutreffend korrigiert

### Referenzen

- ARCH-AUDIT-0002, Kapitel 14.4, Maßnahme N4

### Breaking Changes

Keine.

### Autor

Platform Director

---

## [1.2.0] — 2026-08-02

### Hinzugefügt

- `Validators/baseValidator.ts` — erste generische Validator-Basisklasse im gesamten Repository ( `abstract class Validator<TTarget, TFinding>` )

- `Validators/traceabilityMatrixValidator.ts` — erste konkrete Implementierung, wrappt `Core/orphanDetector.ts` (Konstruktor-Injektion fuer Testbarkeit)
- `Services/runTraceability.ts` : sechs reale Events ueber den Enterprise Event Bus ( `publishTraceabilityEvent()` , best-effort) - `TraceabilityBuildStartedEvent`, `TraceabilityBuildCompletedEvent`, `TraceabilityBuildFailedEvent`, `CoverageCalculatedEvent`, `OrphanDetectedEvent`, `TraceabilityReportGeneratedEvent`
- `tests/unit/traceabilityValidator.test.ts`

## Geändert

- `src/platform/EventMesh/Events/StandardEventCatalog.ts` : vier fehlende Traceability-Events ergaenzt (nur `TraceabilityBuildCompletedEvent` war zuvor katalogisiert)
- `Services/runTraceability.ts` nutzt jetzt `TraceabilityMatrixValidator` statt `OrphanDetector` direkt zu instanziiieren
- `manifest.json` , `README.md` , `component.yaml` : Implementierungsstand Stufe 3 (teilweise: Publish-Seite umgesetzt, keine Subscriptions) und Stufe 4 (umgesetzt) aktualisiert

## Referenzen

- ARCH-AUDIT-0002, Kapitel 14.4, Massnahme N4 (Folgearbeit)

## Breaking Changes

Keine.

## Autor

Platform Director

---

## Hinweis zur Erstfassung

Diese Datei ist zusammen mit `component.yaml` die **erste ihrer Art im Repository**.

Die 23 bestehenden Komponenten führen weder `CHANGELOG.md` noch `component.yaml` — dokumentiert als GAP-013 und GAP-014 in `docs/architecture/ARCHITECTURE_GAP_REPORT.md` sowie als Regeln `GOV-REPO-002` und `GOV-REPO-004` in `ESS-0012-CONTRACTS`.

Diese Komponente dient insoweit als Referenzimplementierung des Metadata Contracts.

# Traceability

Quelle: `src/platform/Traceability/README.md`

## Traceability

### Enterprise Component

Status: Development

Version: 1.0.0

Layer: Querschnittsmodul mit erweitertem Zugriff

Owner: Platform Director

### Purpose

Die Enterprise Traceability Matrix (ETM) verwaltet die bidirektionalen Beziehungen zwischen sämtlichen Enterprise-Artefakten.

Sie beantwortet lückenlos und maschinell:

Welche Anforderung, welche Entscheidung, welche Regel führte zu welchem Artefakt — und welches Artefakt erfüllt welche Regel?

Sie erzeugt keine Regeln, keine Dokumentation und keine Artefakte.

Sie verknüpft ausschließlich bereits vorhandene, validierte Enterprise-Artefakte.

### Verhältnis zur Spezifikation

Diese Komponente **implementiert** ESS-0011.

| Dokument                        | Rolle                                                     |
|---------------------------------|-----------------------------------------------------------|
| ESS-0011                        | Spezifikation der ETM — Architektur, Prozesse, Reports    |
| ESS-0011-CONTRACTS              | Regelwerk — Link Contract, Coverage, Orphans              |
| <code>docs/traceability/</code> | operative Dokumentation — Betrieb, Lifecycle, Validierung |
| <b>diese Komponente</b>         | <b>Implementierung</b>                                    |

Die Spezifikation wird hier **nicht wiederholt**. Bei Abweichungen gilt ESS-0011.

## Struktur

|               |                                   |
|---------------|-----------------------------------|
| Traceability/ |                                   |
| Contracts/    | komponentenspezifische Contracts  |
| Core/         | Basislogik der Matrix             |
| Discovery/    | Erkennung verknüpfbarer Artefakte |
| Events/       | Event-Erzeugung und -Konsum       |
| Interfaces/   | öffentliche Schnittstellen        |
| Models/       | Datenmodelle                      |
| Registry/     | Registrierung der Matrixeinträge  |
| Reports/      | Berichtsdatenerzeugung            |
| Services/     | Ausführung und Orchestrierung     |
| Validators/   | Prüfung der Matrix                |
| Versioning/   | Versionierung der Matrix          |

Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß ESS-0001-CONTRACTS Chapter 3.

## Traceability-Achsen

Die ETM verknüpft sieben Artefaktklassen:

ESS · ADR · Exception · Component · Interface · Event · Test

**Vorwärts** — Regel zu Umsetzung: ESS-Kapitel → ADR → Component → Interface → Event → Test

**Rückwärts** — Umsetzung zu Regel: Test → Event → Interface → Component → ADR → ESS-Kapitel

Eine Verknüpfung, die ausschließlich in einer Richtung existiert, ist ein Befund.

## Abhängigkeiten

**Zulässig:** Core, Shared, Registry, Knowledge, Discovery, Documentary

**Unzulässig:** Supervisor, PlatformDirector

Die Kommunikation nach oben erfolgt ausschließlich über Enterprise Events.

Die Einordnung als Querschnittsmodul mit erweitertem Zugriff folgt der bereits für Release etablierten Regelung aus ESS-0001-CONTRACTS Chapter 16 (Cross Cutting Modules) und erfordert keine Änderung der Layer-Hierarchie.

## Rollentrennung

| Instanz                   | Rolle                      |
|---------------------------|----------------------------|
| <b>Traceability (ETM)</b> | <b>verknüpft und misst</b> |
| Governance Validator      | prüft Regeln               |
| Supervisor                | bewertet und eskaliert     |

| Instanz           | Rolle                        |
|-------------------|------------------------------|
| Platform Director | entscheidet                  |
| Version Manager   | leitet Versionsauswirkung ab |

Die ETM stellt Beziehungen fest. Sie bewertet nicht und entscheidet nicht.

---

## Datenquellen

Die ETM erzeugt keine eigenen Daten. Sie liest ausschließlich:

Knowledge Graph · Enterprise Registry · Digital Twin · ESS Registry · Exception Registry ·  
docs/adr/ · Validierungsergebnisse

Fehlt eine Quelle, meldet die ETM dies als Befund und arbeitet nicht mit Annahmen weiter.

---

## Ablage der Matrix

```
.ai/knowledge/traceability/  
matrix.json      vollständige Matrix  
coverage.json    Abdeckungskennzahlen  
orphans.json     nicht verknüpfte Artefakte
```

Diese Dateien werden ausschließlich generiert und niemals manuell bearbeitet.

---

## ESS Reference

ESS-0011 — Enterprise Traceability

ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

ESS-0010 — Documentary Engine

ESS-0001-CONTRACTS — Master Enterprise Standard

---

## ADR References

ADR-0015 — Einführung der Traceability-Komponente

ADR-0013 — ESS Documentation Responsibility Consolidation

ADR-0014 — Documentation Governance Validator

---

## Implementierungsstand

Diese Komponente ist **teilweise implementiert** (ARCH-AUDIT-0002, N4/N4-Folge, 2026-08-02).

| Stufe | Voraussetzung                         | Zustand                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | JSON-Schemata, vollständige Metadaten | <b>umgesetzt</b> — <code>Models/traceabilityModels.ts</code>                                                                                                                                                                                                                                                                                                                                                                                                |
| 2     | Core, Interfaces, Models, Registry    | <b>umgesetzt</b> — <code>Core/</code> , <code>Interfaces/index.ts</code> , <code>Registry/traceabilityRegistry.ts</code>                                                                                                                                                                                                                                                                                                                                    |
| 3     | Enterprise Event Bus                  | <b>teilweise umgesetzt</b> — jeder Lauf veröffentlicht sechs reale Events über <code>src/platform/EventMesh/Core/EventBus.ts</code> ( <code>publishTraceabilityEvent()</code> in <code>Services/runTraceability.ts</code> , best-effort). Nur die Publish-Seite; die fünf <code>consumes</code> -Events aus dem Manifest werden nicht abonniert.                                                                                                            |
| 4     | Validator-Basisklasse                 | <b>umgesetzt</b> — <code>Validators/baseValidator.ts</code> (abstract class <code>Validator&lt;TTarget, TFinding&gt;</code> ), erste generische Validator-Basis im gesamten Repository. <code>Validators/traceabilityMatrixValidator.ts</code> ist die erste konkrete Implementierung und wrappt <code>Core/orphanDetector.ts</code> ; <code>Services/runTraceability.ts</code> nutzt sie jetzt statt <code>OrphanDetector</code> direkt zu instanziiieren. |

Aufruf: `npm run traceability:build` (`Services/runTraceability.ts`). Baut die Matrix, schreibt `matrix.json` / `coverage.json` / `orphans.json` unter `.ai/knowledge/traceability/` sowie `docs/traceability/COVERAGE_REPORT.md`, und führt das `tests`-Feld in allen Plattform-Manifesten anhand real gefundener Testdateien nach. Kein automatischer Lauf beim Serverstart — bewusst, siehe AUD2-F-014 (unerwünschte Dokumentmutation bei jedem Start).

**Korrektur zur ursprünglichen Einschätzung:** Die zuvor hier genannte Blockade — ohne Knowledge Graph und Digital Twin keine Datengrundlage (GOV-KG-001, GOV-TWIN-001) — traf für diesen ersten Ausbau nicht zu. Die Achsen ESS↔Component und Component↔Test kommen ohne Knowledge Graph und Digital Twin aus; sie lesen direkt `.ai/registry/ess-registry.json`, `docs/adr/adr_history.json`, `src/platform/*/manifest.json` und `tests/`. Die ADR- und Event-/Interface-Achsen aus Kapitel „Traceability-Achsen“ sind noch nicht als geprüfte Verknüpfung abgebildet (siehe `Core/traceabilityBuilder.ts`, Kommentar am Dateikopf) — dafür fehlt tatsächlich eine zweite, unabhängige Quelle je Achse.

---

## Notes

Vollständige Dokumentation unter `docs/traceability/`.

Diese Komponente ist die erste im Repository mit vollständigem Metadatensatz einschließlich `component.yaml` und `CHANGELOG.md`.

# Validators

---

Quelle: `src/platform/Validators/README.md`

## Validators

### Enterprise Component

Status: Unspecified

Version: 1.0.0

Owner: CAPITAL-AI

---

### Purpose

Fuer diese Komponente existiert keine funktionale Spezifikation. Der urspruengliche, vom Enterprise Bootstrapper generierte Platzhaltertext ("Describe the responsibility of the Validators component.") wurde entfernt, um keine nicht existierende Spezifikation vorzutaeuschen (No-Demo-Data-Policy, docs/DATENSCHUTZ\_PROTOKOLL.md).

---

### ESS Reference

ESS-0001

ESS-0001-CONTRACTS

---

### ADR References

None

---

### Dependencies

Keine - es existiert kein Code.

---

### Events

Keine - es existiert kein Code.

---

## Notes

ARCH-AUDIT-0002 (J5, 2026-08-02): fuer diese Komponente existiert ausser dem vom Enterprise Bootstrapper generierten Platzhaltertext keine funktionale Spezifikation und kein Code. Status auf 'unspecified' korrigiert, um den Zustand ehrlich abzubilden - vorher suggerierte 'development' aktive Arbeit, die nicht stattfindet. Die Komponente kann bei Bedarf zu einem spaeteren Zeitpunkt spezifiziert und implementiert werden.

# VersionManager

---

Quelle: `src/platform/VersionManager/README.md`

## VersionManager

### Enterprise Component

Status: Implemented

Version: 1.1.0

Owner: CAPITAL-AI

---

### Purpose

Verwaltet die verbindliche Plattformversion und Build-Nummer von CAPITAL-AI. Fuehrt admin-getriggerte Versions-Bumps (patch/minor/major) aus, schreibt Git-/Docker-Tags fort und generiert dabei 11 Compliance-Markdown-Dokumente unter `docs/` (CHANGELOG, RELEASE\_NOTES, ARCHITECTURE\_REPORT, u.a.).

Mit Version 1.1.0 erweitert der **Naming & Repository Convention Validator** die Komponente um eine deterministische, read-only Repository-Pruefung. Sie validiert Projektidentitaet, Semantic-Version-Synchronitaet zwischen `package.json` und `package-lock.json`, ADR-/ESS-/React- und Plattform-Namenskonventionen, case-insensitive Pfadkollisionen, Legacy-Identitaeten sowie abgelaufene Enterprise Exceptions.

Persistenz: `uploads/version_manager.json`. REST-API: GET `/api/admin/version`, POST `/api/admin/version/bump` (admin- und Step-Up-geschuetzt, siehe `server/iam/`).

Repository-Validation:

- `npm run repository:validate` — Strict Mode fuer gezielte Governance-/Quality-Pruefungen.
- `npm run repository:validate:advisory` — Advisory Mode; Findings werden gemeldet, aber nicht blockiert.
- `npm run predeploy:check` — prueft ausschliesslich technische und sicherheitsrelevante Deployment-Readiness und ruft den Repository Convention Validator bewusst nicht auf.

Der Repository Convention Validator ist damit **kein Deployment-Gate**. Governance-, Naming-, ADR-/ESS- und Repository-Policy-Findings werden ausserhalb der technischen Live-Deployability- Pipeline behandelt. Die Deployment-Pipeline beantwortet ausschliesslich, ob ein Build technisch und sicher in der Live-Umgebung betrieben werden kann.

Der Validator veraendert keine Dateien und genehmigt keine Ausnahmen. Abweichungen werden nur als Findings ausgegeben; Ausnahmen bleiben ausschliesslich der Governance bzw. dem Platform Director vorbehalten.

---

## ESS Reference

ESS-0001

ESS-0001-CONTRACTS

ESS-0004 — Enterprise Version Manager

---

## ADR References

ADR-0011 — Bestandsschutz / Enterprise Exception Registry

ADR-0020 — Naming & Repository Convention Validator Capability

---

## Dependencies

`server/systemEvents.ts`, `server/documentHygiene.ts`, `server/iam/authMiddleware.ts`, `server/iam/types.ts`.

Die Convention-Validation selbst verwendet ausschliesslich Node.js `fs / path` und ist damit ohne weitere Runtime-Abhaengigkeiten ausfuehrbar.

---

## Events

Produziert aktuell keine Enterprise-Bus-Events (ESS-0013). Protokolliert stattdessen ueber das aeltere `logSystemEvent()`-Log. Die zuvor im `manifest.json` genannten Events (Version-CalculatedEvent u.a.) waren nie implementiert und wurden entfernt (ARCH-AUDIT-0002 J5).

---

## Notes

ARCH-AUDIT-0002 (J5, Kapitel 14.6): physisch aus `server/versionManager.ts` hierher verschoben. Vorher deklarierte die ESS-Registry ( `implementedBy` ) diesen Ordner fuer ESS-0004, obwohl er nur eine leere, vom Enterprise Bootstrapper generierte Huelle war und der tatsaechliche Code unter `server/` lag - eine Governance-Inkonsistenz, die mit dieser Verschiebung behoben wurde.

ADR-0020 erweitert die bestehende Komponente bewusst um eine Validator-Capability statt einen parallelen Naming-Agenten einzufuehren. Damit bleibt Versionierungs- und Repository-Konformitaet in einer verantwortlichen Enterprise-Komponente gebuendelt, ohne das technische Deployment-Gate mit Governance-Policies zu vermischen.

# Changelog

---

Quelle: `src/platform/Vocabulary/CHANGELOG.md`

## Changelog

### 1.0.0 — 2026-08-09

#### Deutsch

- Canonical Vocabulary Registry eingefuehrt.
- Typed Concept Model nach ESS-0017-CONTRACTS angelegt.
- Deterministische Validierung fuer Concept IDs, technische Begriffe und Authorities implementiert.
- Alias- und Display-Term-Kollisionen werden blockiert.
- DE/EN-Aufloesung und Forbidden-Term-Erkennung implementiert.
- Initiale Seed-Concepts fuer Billing, Analytics und Vocabulary Governance hinzugefuegt.
- Contract-Tests hinzugefuegt.
- Keine aktiven Code-Renames und keine neuen Event-Typen.

#### English

- Introduced the Canonical Vocabulary Registry.
- Added the typed concept model defined by ESS-0017-CONTRACTS.
- Added deterministic validation for concept IDs, technical terms and authority references.
- Added alias/display-term collision protection.
- Added DE/EN resolution and forbidden-term detection.
- Added initial seed concepts for billing, analytics and Vocabulary Governance.
- Added contract tests.
- No active code renames and no new event types.

# CAPITAL-AI Canonical Vocabulary Registry

Quelle: `src/platform/Vocabulary/README.md`

## CAPITAL-AI Canonical Vocabulary Registry

Authority: `ESS-0017` / `ESS-0017-CONTRACTS`

Architecture decision: `ADR-0046`

### Deutsch

Die Vocabulary-Komponente ist die kanonische Terminologiequelle fuer CAPITAL-AI. Sie verwaltet stabile Concept-IDs, englische technische Begriffe, deutsche und englische Anzeigeformen, Aliase sowie verbotene Begriffe.

Phase 2 veraendert keine bestehenden Code-Namen. Ein bestehender Identifier wird erst in Phase 3 nach Safe-Rename-Impact-Analyse geaendert.

Die Registry erzeugt keinen zweiten Event Bus, Knowledge Graph oder Traceability Store. Phase 2 enthaelt absichtlich keine neuen Event-Namen; die spaetere EventMesh-Anbindung darf erst nach Abgleich mit dem kanonischen EventCatalog erfolgen.

### Verwendung

```
import { createDefaultVocabularyRegistry, VocabularyService } from './index';

const registry = createDefaultVocabularyRegistry();
const vocabulary = new VocabularyService(registry);

vocabulary.getCanonicalTerm('Abonnement'); // Subscription
vocabulary.getDisplayName('Subscription', 'de'); // Abonnement
```

### Validierung

Contract-Test:

```
npx tsx src/platform/Vocabulary/Tests/vocabularyRegistry.test.ts
```

Geprueft werden unter anderem Concept-ID-Format, technische Begriffe, Authority-Referenzen, Alias-Kollisionen, DE/EN-Aufloesung und Immutable Snapshots.

### English

The Vocabulary component is CAPITAL-AI's canonical terminology source. It manages stable concept identities, English technical terms, German and English display mappings, aliases, forbidden terms and deterministic collision detection.

Phase 2 does not rename existing runtime identifiers. Active renames are deferred to the Safe Rename Gate in Phase 3.

The component does not introduce a second Event Bus, Knowledge Graph or Traceability store. No new event type is registered in Phase 2; EventMesh integration requires canonical EventCatalog compatibility validation first.